

STEP 1

Analyzing the Design Mockup

- 1-1 First impressions of the design mockup
- 1-2 Coding process and plan
- 1-3 Splitting the page into parts
- 1-4 Checking the parts flow (eye flow)
- 1-5 Planning how to mark up in HTML
- 1-6 Planning how to manage CSS

1-1

First impressions of the design mockup

This project mockup shows a company website design that seems neat, simple, and trustworthy.

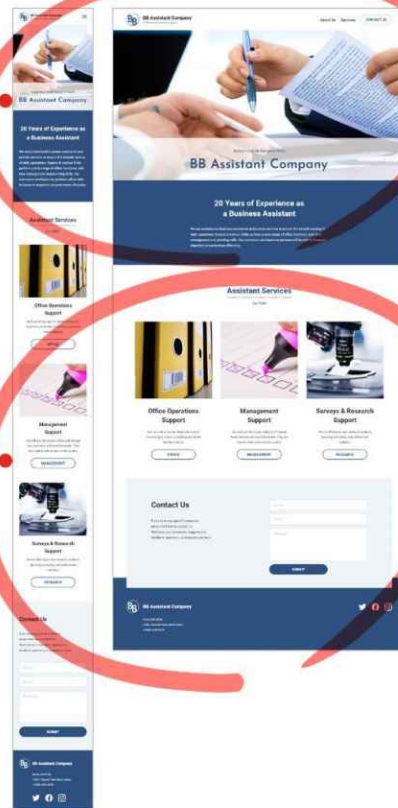
Parts at the top grow and shrink depending on the browser window size. There is not much of a difference between the mobile and desktop layout. So it seems easy to achieve this just by setting them as a fluid width layout.

However, note that we need to resize the font and space in responsive.

Parts at the bottom are aligned vertically on mobile and horizontally on desktop. To achieve this, make them fluid width and change alignment direction at a specific breakpoint.

This project specification does not define any breakpoints. Best breakpoints should be determined by how we adjust these parts to various browser window sizes.

Based on the above first impressions, we are going to prepare for coding.



1-2

Coding process and plan

We'll code the page as following steps.



Split the page on mockup into parts.

Mark them up with HTML and CSS.

Combine them and construct the page.

Basic coding plan

We'll code according to the design specification. For matters not in it:

- ✓ Make it mobile-first.
- ✓ Create each part individually.
- ✓ Handle the code separately per part.
- ✓ Also, manage responsive settings per parts basis.

Part-based development has the advantage of making code simpler and editing easier.

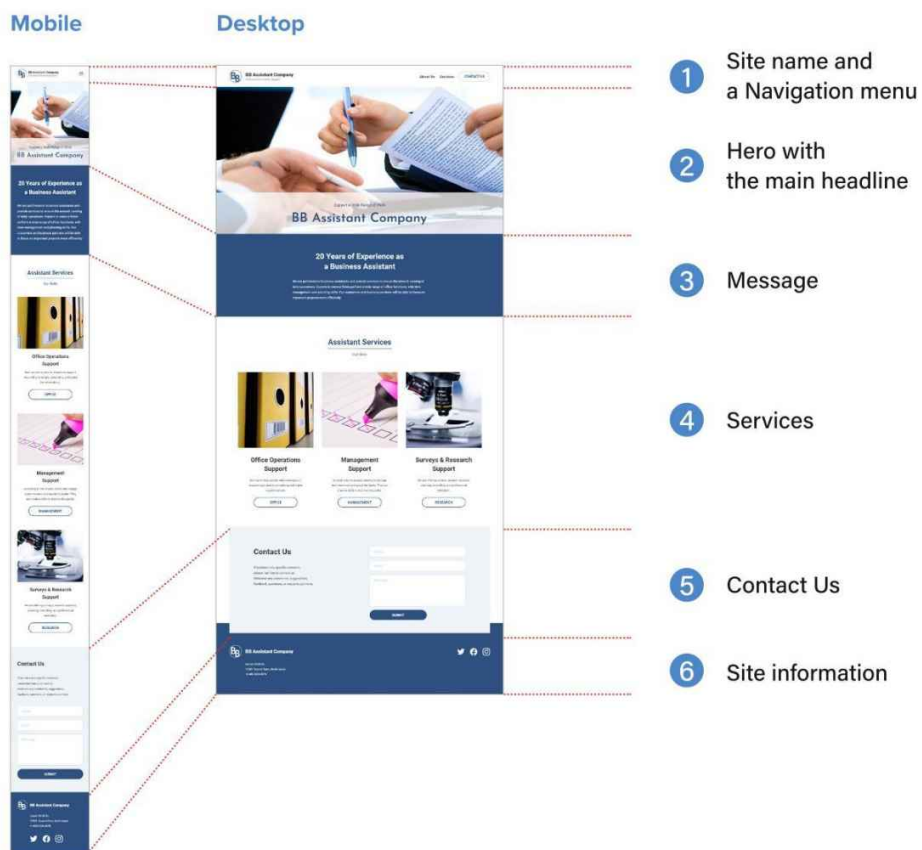


1-3

Splitting the page into parts

The first step is to split the page into parts. The page design shows the division of parts visually. By drawing boundaries, we can split it into six parts. There is no difference between mobile and desktop.

The contents of each part are as below.

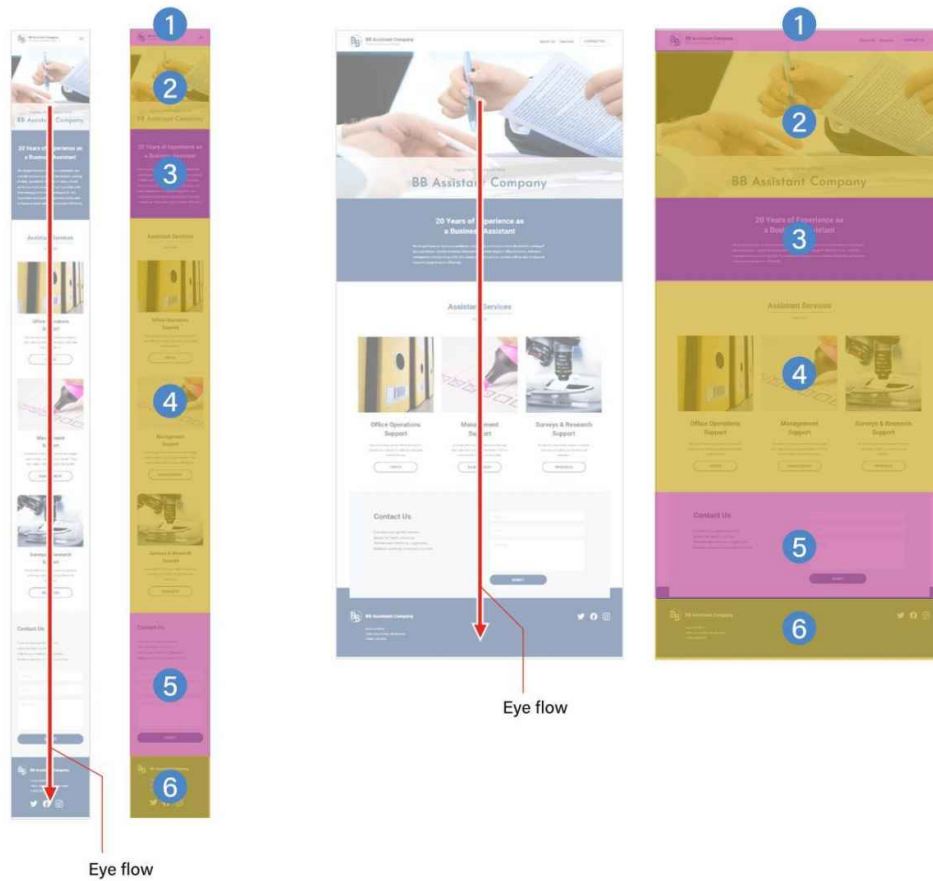


1-4

Checking the parts flow (eye flow)

We check the designed eye flow pattern for the ①-⑥ parts. Readers will scan from top to bottom following ① - ⑥ lined up vertically.

So, we'll make the ① - ⑥ code order the same as the eye flow.



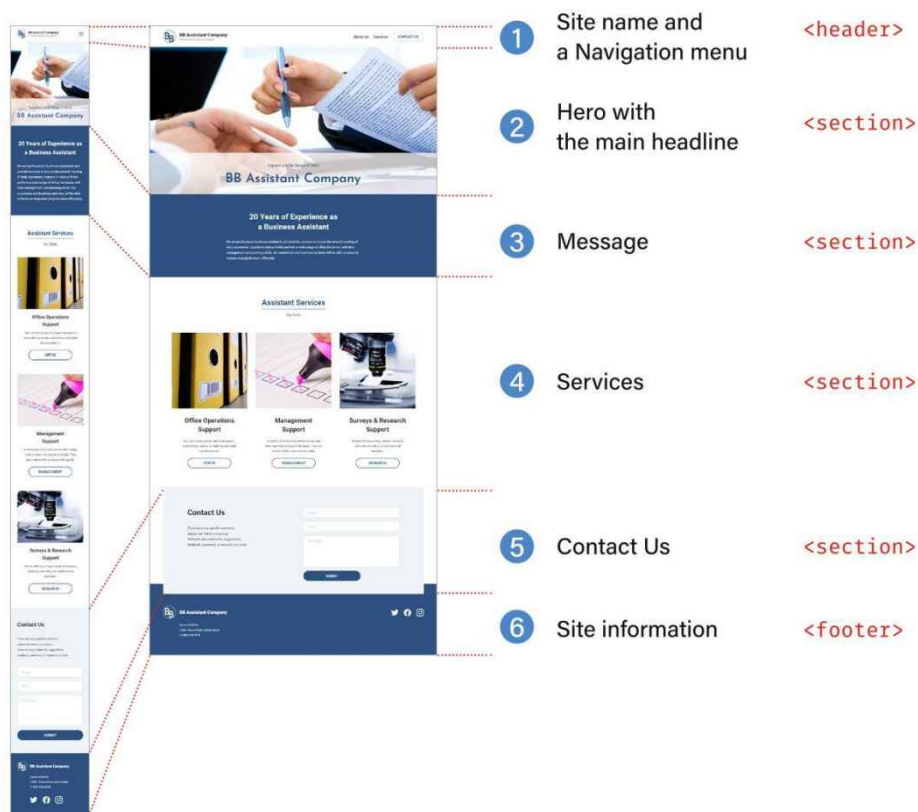
1-5

Planning how to mark up in HTML

To define the meaning and structure of ① - ⑥ parts, determine how to mark them up in HTML as clearly as possible.

We'll use:

- ✓ **<header>** to define ① as a header that appears at the top of the page.
- ✓ **<footer>** to define ⑥ as a footer that appears at the bottom of the page.
- ✓ **<section>** to define ② - ⑤ as a section that consists main contents.



Heading level

In HTML, the heading has <h1>-<h6> levels and defines the whole structure of the web page. According to this, we detect the heading of each part and determine their levels.

We'll use:

- ✓ <h1> to define the main headline over the hero image as the top-level heading.
- ✓ <h2> to define the heading of ③ - ⑤ as one level lower heading.

```
<body>
<header>Site name and Navigation</header>

<section>
  <h1>Hero's main headline:
  BB Assistant Company</h1>
</section>

<section>
  <h2>Heading of ③ :
  20 Years of Experience as
  a Business Assistant</h2>
</section>

<section>
  <h2>Heading of ④ :
  Assistant Services</h2>
</section>

<section>
  <h2>Heading of ⑤ :
  Contact Us</h2>
</section>

<footer>Site information</footer>
</body>
```

Document Structure

The result will be a simple document structure, as shown on the right.

This document structure is a heading-level outline. For more details, please refer to STEP 10-5. We'll check the structure after the page is complete.



Site name and Navigation

Hero's main headline

Heading of ③

Heading of ④

Heading of ⑤

Site information

Class name

To identify each part and style it with CSS, we need to give it a class name. There are various naming conventions, but here we specify a simple name that represents the part.



```
<body>
<header class="header">
  Site name and Navigation
</header>

<section class="hero">
  <h1>Hero's main headline:
  BB Assistant Company</h1>
</section>

<section class="message">
  <h2>Heading of ③:
  20 Years of Experience as
  a Business Assistant</h2>
</section>

<section class="service">
  <h2>Heading of ④:
  Assistant Services</h2>
</section>

<section class="contact">
  <h2>Heading of ⑤:
  Contact Us</h2>
</section>

<footer class="footer">
  Site information
</footer>
</body>
```

Now the markup plan is ready. We will mark up the details at each step of the parts creation.

1-6

Planning how to manage CSS

Generally, the page design and layout have some consistent patterns. We check and find them from the mockup, then consider how to manage their settings. There are ways to handle them together as a basic setting for the whole page or individually as each part's setting.

Content width with left and right space

The first checkpoint is the content width with left and right space. Drawing lines on the left side of the header's site name and the right side of the navigation menu makes clear that:

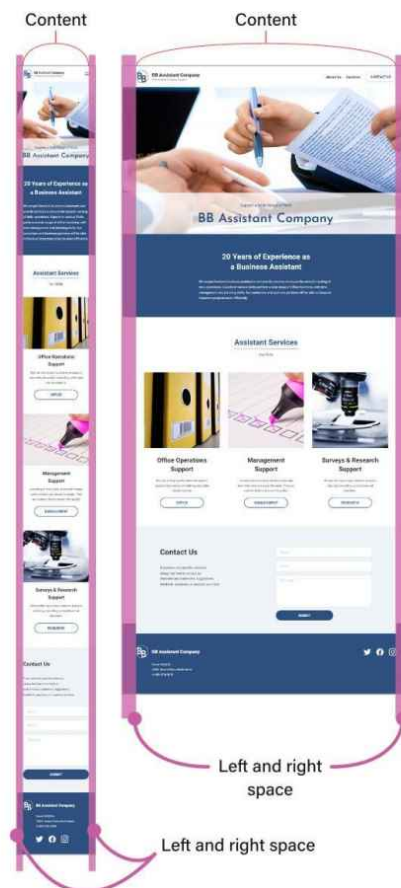
- ✓ Each part has consistent left and right space.
- ✓ The contents are centered.
- ✓ The above applies to both mobile and desktop layouts.

Also, the design specification says:

Make it fluid width but limit the maximum 1240px.

It requires adjusting content width fluidly to fit the browser window size and setting its max-width to 1240px.

So we'll manage these widths and spaces as a basic setting for the whole page and make them easier to adjust.



Top and bottom space

The second checkpoint is the top and bottom space for each part. It seems consistent, but its sizes are different except for mobile's ③ and ④.

Also, each space is larger on desktop than on mobile, but there is no notable correlation between them.

We'll manage these spaces as the individual setting for each part.



Typography

The third checkpoint is typography: font family, size, weight, color, and so on. For example, those settings for the `<h2>` level heading (**red**) and body text (**blue**) in the mockup are as follows. For desktop, we are listing only differences from mobile to implement with a mobile-first approach.



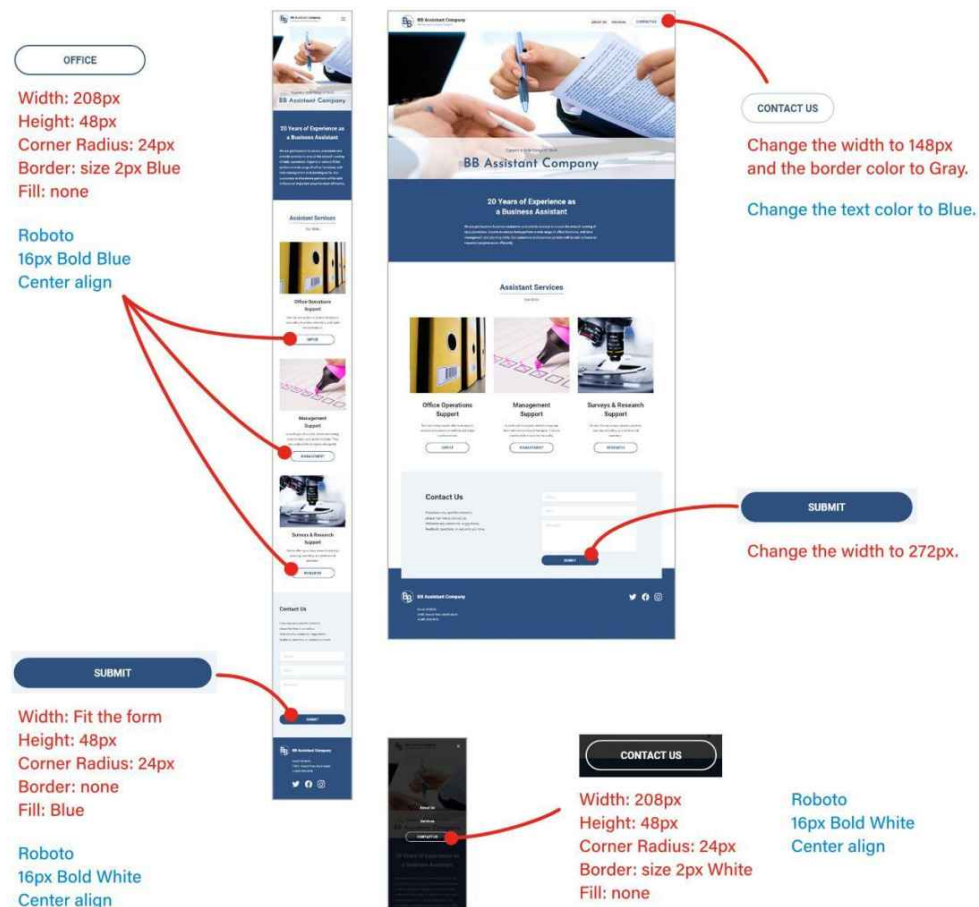
For headings (**red**), font family, size, and weight are the same; color, alignment, decoration, and others are not. For body text (**blue**), only font family and weight are the same.

Accordingly, we'll manage the font family as a basic setting for the whole page, others as the individual setting for each part.

Buttons

The last checkpoint is buttons which seem to use some consistent style. For button shape (**red**), height and corner radius are the same; width, color, and others are not. For button text (**blue**), font family, size, weight, and others are the same except the color.

We'll manage buttons as the basic setting to use throughout the page.



Analyzing the design mockup is complete. In the next step, we'll start coding.

STEP 2

Start Coding

- 2-1 Preparing HTML, CSS, Image files
- 2-2 Previewing the page

2-1

Preparing HTML, CSS, Image files

We prepare HTML, CSS, and image files for coding.

Preparing images

First, prepare images for the website. In this project, we use the ones provided with the design mockup.

There is no favicon image, but the SVG logo (bb.svg) is suitable as an alternative. PNG logo (bb.png) created with converting from the SVG is for some browsers not supporting SVG favicon.



Image created



bb.png
— converted from
bb.svg

Images provided along with the mockup



hero.jpg

BB Assistant Company

hero.svg



bb.svg



bb-white.svg



office.jpg



manage.jpg



research.jpg



logo-twitter.svg



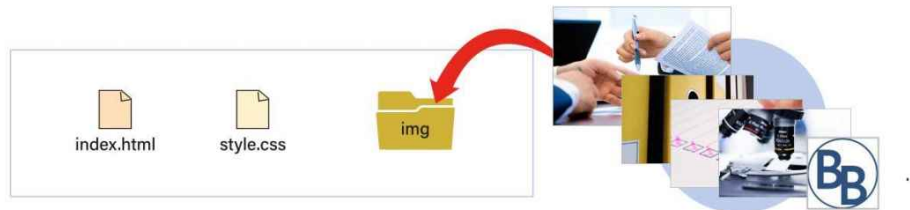
logo-facebook.svg



logo-instagram.svg

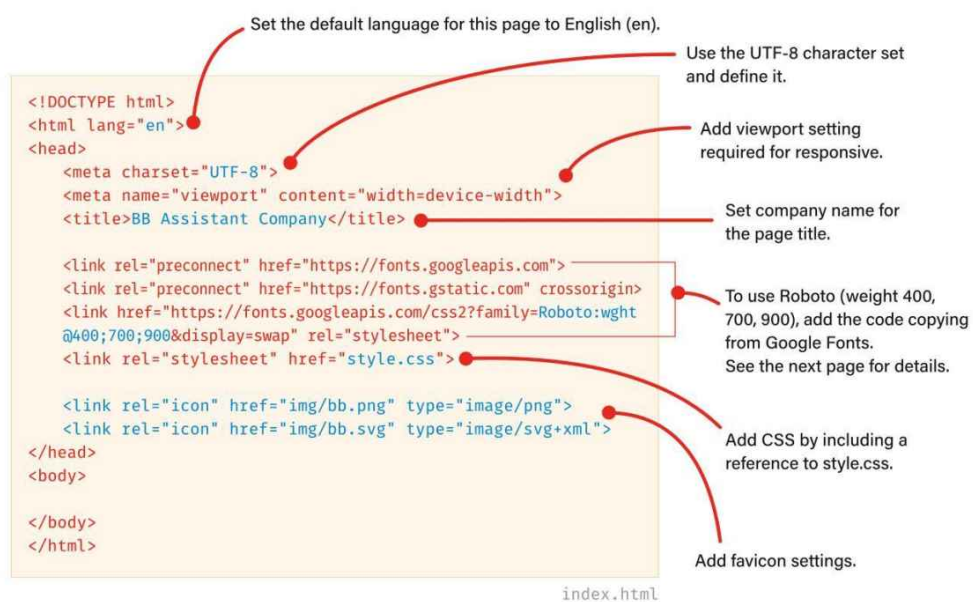
Creating HTML and CSS files

Next, create an HTML file named **index.html** and a CSS file named **style.css**. In addition, make an **img** folder and put all the images in it.



Writing the basic HTML

We write the basic HTML settings in **index.html**.



How to use Google Fonts

Roboto is a font noted in the specification and used in the design mockup. It is available for free in Adobe Fonts and Google Fonts. Adobe fonts require the Adobe ID, so we use Google Fonts.

Checking the text settings in the mockup, they use three weights of Roboto: Regular, Bold, and Black. In Google Fonts, select these weights and copy the code to embed.

Search and open Roboto's page.

Select

- Regular 400
- Bold 700
- Black 900

Copy the code to embed the font.

Use on the web

To embed a font, copy the code into the <head> of your HTML

☒ <link> ☐ @import

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700;900&display=swap" rel="stylesheet">
```

CSS rules to specify families

```
font-family: 'Roboto', sans-serif;
```

Google Fonts
<https://fonts.google.com/>

Writing the basic CSS

We write the basic CSS settings in **style.css**.

Basic text settings

Add font-family settings as decided in STEP 1-6 *Typography*. To use Roboto throughout the page, set the font-family of <body> as follows.

Reset settings

Add some CSS to clear the browser's default formatting of HTML elements defined by its built-in UA stylesheet.

Such kinds of CSS are called Reset CSS, Normalize CSS, Sanitize CSS, etc. There are various systems and methods, but we set them as below in this project.

These settings remove the default margin and padding.

In this project, space sizes around elements seem various. So we start styling with no space around the major elements: body, h1-h6, p, ul, and figure.

These settings make the link color the same as its parent, with no underline.

When mousing over a link, it will become a little bit transparent.

That's all basic CSS settings.

```
@charset "UTF-8";

/* Basic */
body {
    font-family: 'Roboto', sans-serif;
}

/* Reset / Normalize / Sanitize */
body, h1, h2, h3, h4, h5, h6, p, ul, figure {
    margin: 0;
    padding: 0;
}

a {
    color: inherit;
    text-decoration: none;
}

a:hover {
    opacity: 0.8;
}
```

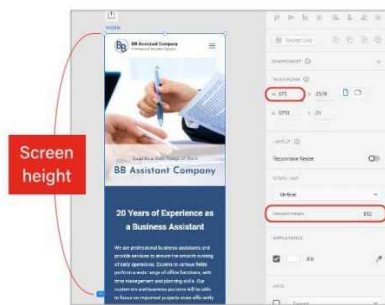
style.css

2-2

Previewing the page

Screen size for mobile and desktop

While making a web page, it needs to check how it looks on various screen sizes (viewport/browser window sizes). We mainly use the same screen size as the design mockup. It's 375 × 812 pixels (iPhone X/XS/13 mini) for mobile and 1366 × 1024 pixels (iPad Pro 12.9 inch) for desktop.



Screen size for mobile:
375 × 812 pixel

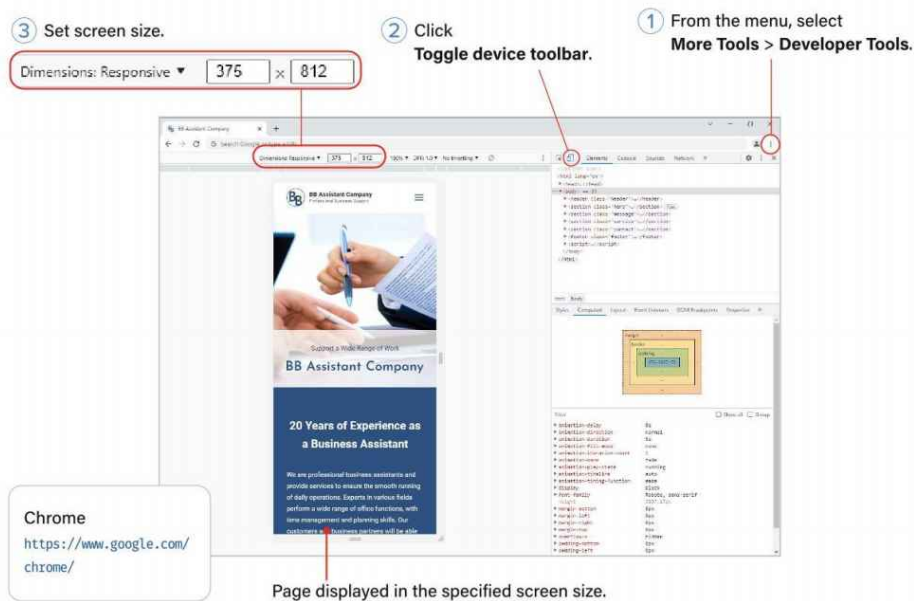
By selecting an artboard in the XD, you can check the screen width (W) and height (Viewport Height).

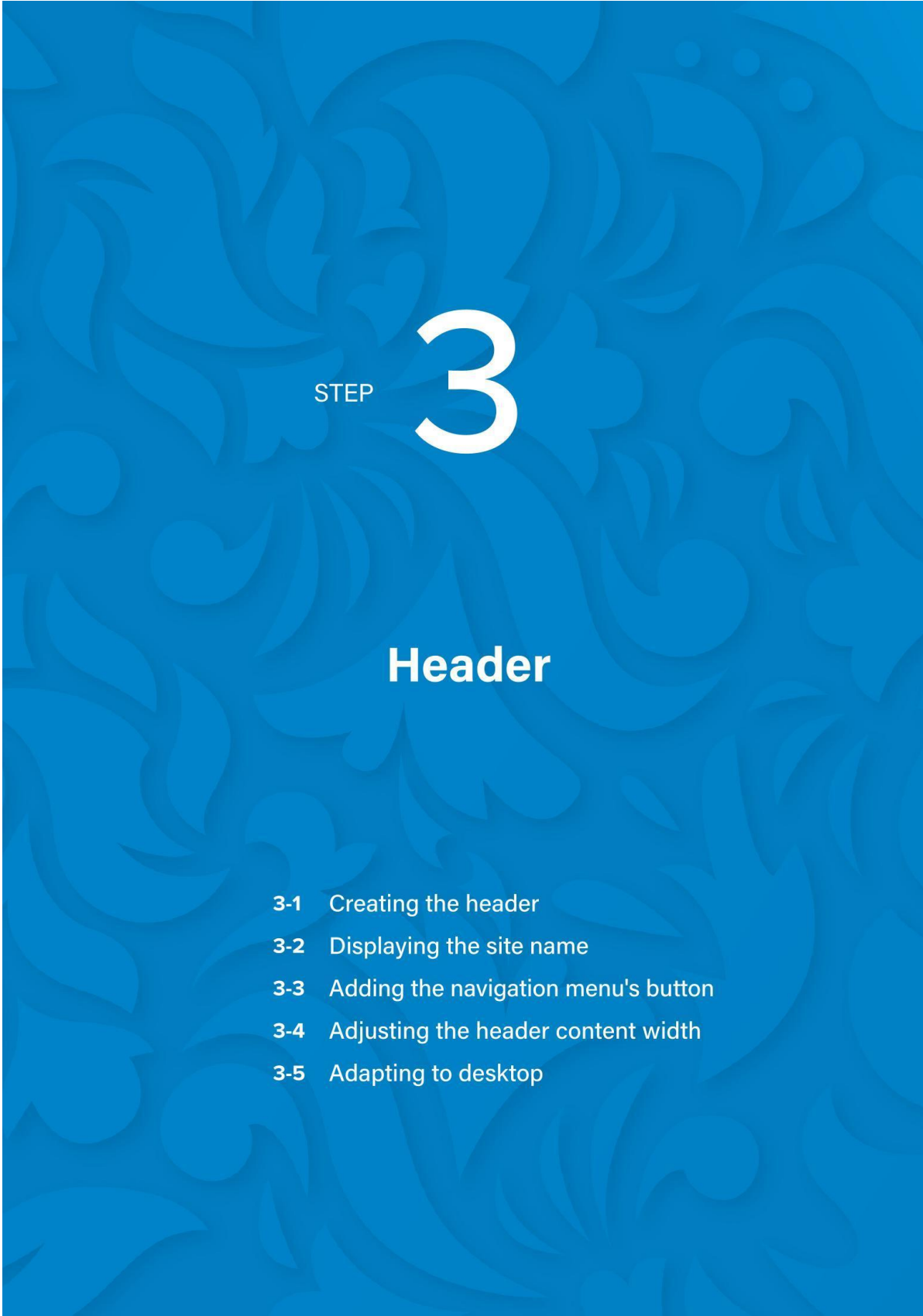


Screen size for desktop:
1366 × 1024 pixels

How to display in specific screen size

To display a web page in specific screen size, use the browser developer tools. Here we use the DevTools in Chrome. It can also show the Flexbox/CSS Grid structure, box structure, and more helpful information for development. We're all set. The next step, start creating the parts.





STEP 3

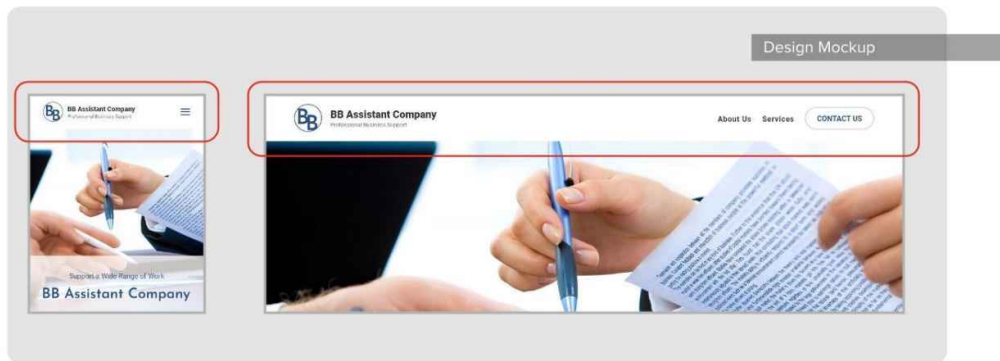
Header

- 3-1 Creating the header
- 3-2 Displaying the site name
- 3-3 Adding the navigation menu's button
- 3-4 Adjusting the header content width
- 3-5 Adapting to desktop

3-1

Creating the header

In this step, we create the ① header part.



Creation Step

The header includes the site name and the navigation menu.

Here we create only the site name and the navigation menu's button. This button is for toggling the menu, but it needs some complex settings to work. So such settings are made in STEP 9.

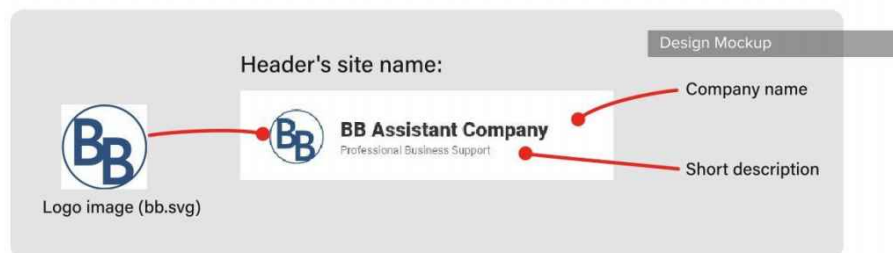


Header to be completed in this step

3-2

Displaying the site name

The site name includes logo image (bb.svg), company name, and short description. The name and description are aligned next to the logo.



As determined in STEP 1-5, we make the header with `<header class="header">` and put the site name in it. The site name uses:

- ✓ `<img>` to place the logo image.
- ✓ `<p>` with a class name to define the company name and description.
- ✓ `<div>` to make a text group of `<p>` elements that Flexbox will align.
- ✓ `<div class="site">` to group image and text as a site name.

```
...  
<body>  
  
<header class="header">  
  <div class="site">  
      
    <div>  
      <p class="name">BB Assistant Company</p>  
      <p class="desc">Professional Business Support</p>  
    </div>  
  </div>  
</header>  
  
</body>
```

index.html



The site name is displayed.

The alt value of the logo image should be the company name. But in this case, we left it empty because of already described by the adjacent text.



Aligning image and text

By using Flexbox, we make image and text aligned horizontally, centered vertically. To achieve this, apply **display: flex** and **align-items: center** to `<div class="site">`. A space between image and text is set 10px with **margin-right** of `<img>`.

```
...
a:hover {
  opacity: 0.8;
}

/* Header */

/* Header's Site Name */
.site {
  display: flex;
  align-items: center;
}

.site img {
  width: 42px;
  margin-right: 10px;
}

.site .name {
  color: #333333;
  font-size: 14px;
  font-weight: 900;
  line-height: 1.2;
}

.site .desc {
  color: #707F89;
  font-size: 10px;
  line-height: 1.2;
}
```

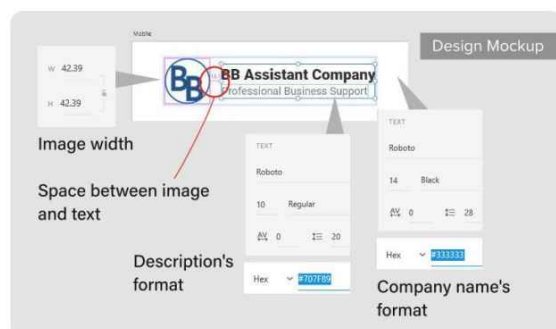
style.css



Set the logo image's width and right margin.

Set the company name's color, font size, weight, and line-height.

Set the description's color, font size, and line-height. The font-weight value will be initial '400' without specified.



Based on the mockup settings, the line-height values would be $28 \div 14 = 2$ and $20 \div 10 = 2$. But spacing gets too wider with those values.



So we set the line-height value 1.2 to closer spacing to mockup look.



3-3

Adding the navigation menu's button

We create the navigation menu's button with `<button>`. To display it as a hamburger button, reset the default browser styles.

For screen readers, we put the text 'MENU' inside the button. `<span class="sr-only">` hides this text from display except for screen readers.

```
<header class="header">
  <div class="site">
    
    <div>
      <p class="name">BB Assistant Company</p>
      <p class="desc">Professional Business Support</p>
    </div>
  </div>

  <button type="button" class="nav-button">
    <span class="sr-only">MENU</span>
  </button>
</header>
```

index.html



The button is displayed.



The button is made invisible by CSS.

The .sr-only class uses the setting of "18F Accessibility Guide" to hide content.

<https://accessibility.18f.gov/hidden-content/>



```
/* Reset / Normalize / Sanitize */
...
a:hover {
  opacity: 0.8;
}

.sr-only {
  border: 0;
  clip: rect(0 0 0 0);
  height: 1px;
  margin: -1px;
  overflow: hidden;
  padding: 0;
  position: absolute;
  width: 1px;
}

/* Header */

/* Header's Site Name */
...
.site .desc {
  color: #707F89;
  font-size: 10px;
  line-height: 1.2;
}

/* Navigation Button */
.nav-button {
  box-sizing: content-box;
  padding: 0;
  outline: none;
  border: none;
  background: none;
}
```

style.css

Creating lines for the hamburger button

The hamburger button style consists of three horizontal lines. We add these lines by CSS and set their weight as 2px, color as blue (primary color), and gap as 4px.



Three horizontal lines are displayed.

```
/* Navigation Button */
.nav-button {
  box-sizing: content-box;
  padding: 0;
  outline: none;
  border: none;
  background: none;
  width: 20px;
  height: 14px;
  cursor: pointer;
  color: #2E5180;
}

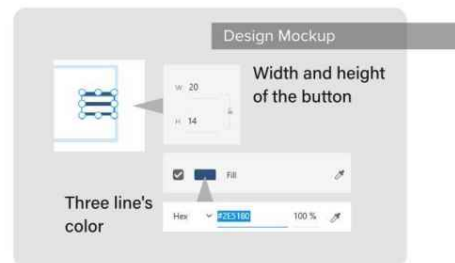
.nav-button::before,
.nav-button::after {
  content: '';
  display: block;
  height: 2px;
  background-color: currentColor;
  transform: translateY(5px);
}

.nav-button::before {
  transform: translateY(-5px);
  box-shadow: 0 6px currentColor;
}
```

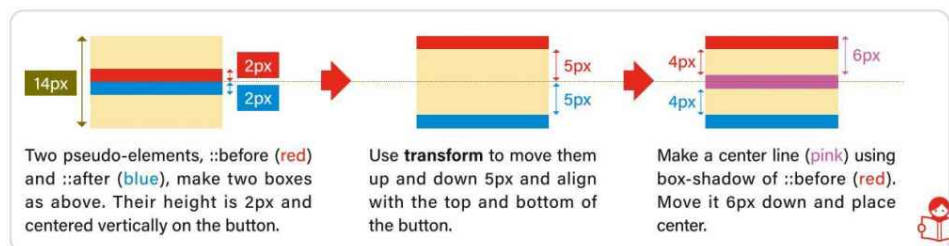
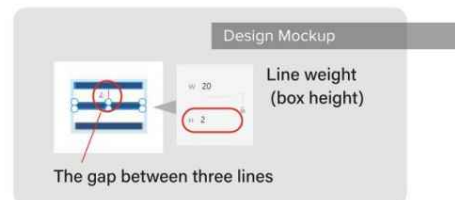
style.css

Make a shadow of the `::before` pseudo-element with `box-shadow` and use it as a center line.

Set the button's width, height, cursor style, and color.



Make top and bottom lines with `::before` and `::after` pseudo-elements of `<button class="nav-button">`. The `height` sets line weight 2px. The `transform` sets line position to make 4px gap.



3-4

Adjusting the header content width

The header content (site name and button) needs adjusting its width. To do so, use `<div class="container">` and apply width setting. We define the width as a basic setting and make it work for adjusting other parts to the same width.

The width value is specified as a percentage to achieve fluid layout. In the mockup, header width is 375px and header content width is 314px. So we set the content width to $314 \div 375 \times 100 \approx 84\%$.

Also, set the left and right margins to `auto` for centering the header content horizontally.

To put the site name on the left side and the button on the right side, apply Flexbox settings. Use `'.header .container'` selector to apply them only to `<div class="container">` in `<header class="header">` and set the header height to 74px.

```
<header class="header">
  <div class="container">
    <div class="site">
      
      <div>
        <p class="name">BB Assistant Company</p>
        <p class="desc">Professional Business Support</p>
      </div>
    </div>
    <button type="button" class="nav-button">
      <span class="sr-only">MENU</span>
    </button>
  </div>
</header>
```

index.html



`<div class="container">` adjusts width, height, and content alignment.



```
/* Basic */
body {
  font-family: 'Roboto', sans-serif;
}

.container {
  width: 84%;
  margin: 0 auto;
}

/* Header */
.header .container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 74px;
}
```

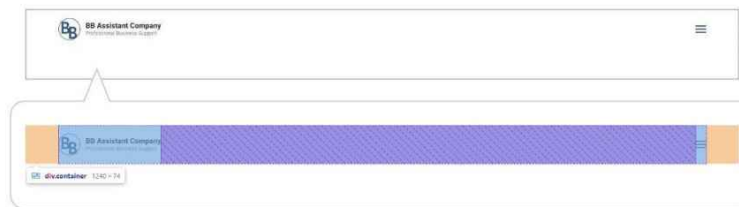
style.css

3-5

Adapting to desktop

We are going to adjust the header to adapt to the desktop screen. However, setting for hiding the button and making the navigation menu responsive will be done at STEP 9.

Adjusting the header content width



In the desktop version of the mockup, the header width is 1366px, and the content width is 1240px. So we set the content width to $1240 \div 1366 \times 100 \approx 91\%$.

Additionally, we set the maximum content width to 1240px according to the design specification. Then set the header height to 100px.

```
/* Basic */
body {
  font-family: 'Noto Sans JP', sans-serif;
}

.container {
  width: 84%;
  margin: 0 auto;
}

.container {
  width: 91%;
  max-width: 1240px;
}

...

/* Header */
.header .container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 74px;
}

.header .container {
  height: 100px;
}
```

style.css



Setting a breakpoint

Settings for the desktop need only be applied when the screen width is larger than a specific breakpoint. But it's not defined in the design specification.

To define the best screen width for a breakpoint, use the ④ part as a guide. Because the ④ is likely to be the first one that breaks down as the screen width decreases.

For example, copy the ④ and decrease its width in the XD. It looks like the layout will not break until about 800px width.

So we set the breakpoint to 900px by considering the above, including the left and right space. This breakpoint is just a temporary value, and we may fix it if it has problems after completing the web page.

```
/* Basic */
body {
  font-family: 'Noto Sans JP', sans-serif;
}

.container {
  width: 84%;
  margin: 0 auto;
}

@media (min-width: 900px) {
  .container {
    width: 91%;
    max-width: 1240px;
  }
}

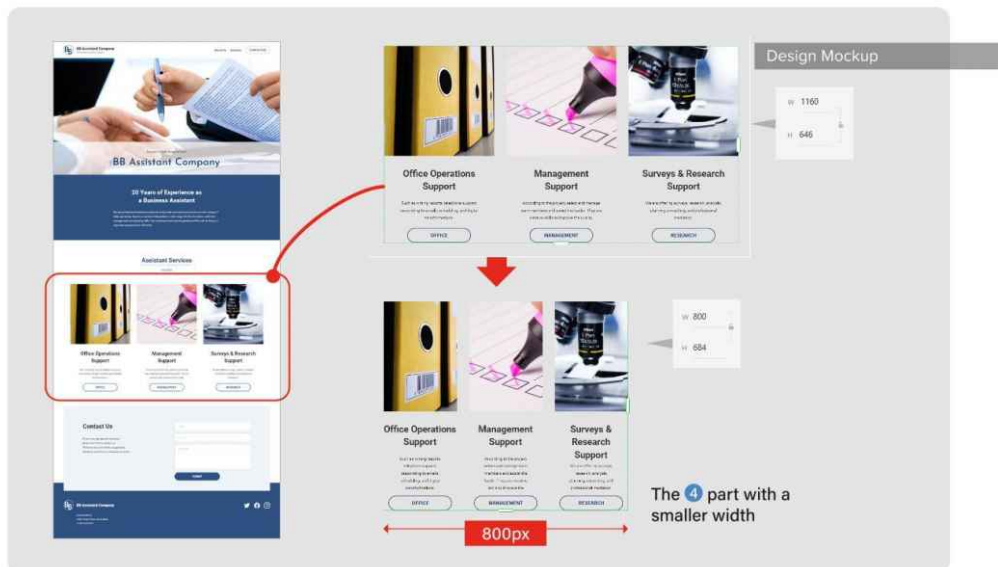
...

/* Header */
.header .container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 74px;
}

@media (min-width: 900px) {
  .header .container {
    height: 100px;
  }
}
```

style.css

Use media query @media to apply settings for desktop only if the screen (viewport) width is 900px or more.



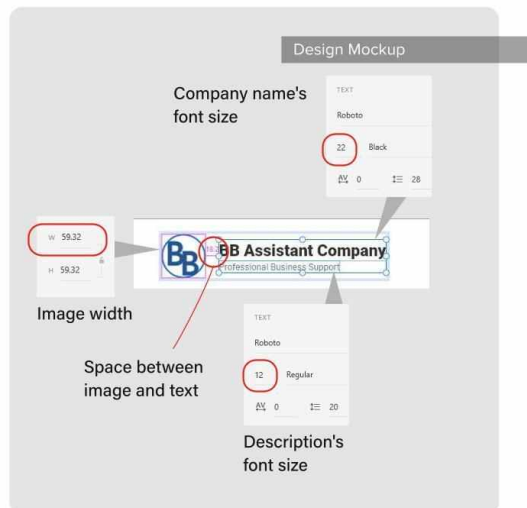
Adjusting the site name

For the desktop version of the site name, increase the image width and the text font size.

Now the header is complete.



The site name is displayed larger.



```
/* Header's Site Name */
.site {
  display: flex;
  align-items: center;
}

.site img {
  width: 42px;
  margin-right: 10px;
}

.site .name {
  color: #333333;
  font-size: 12px;
  font-weight: 900;
  line-height: 1.4;
}

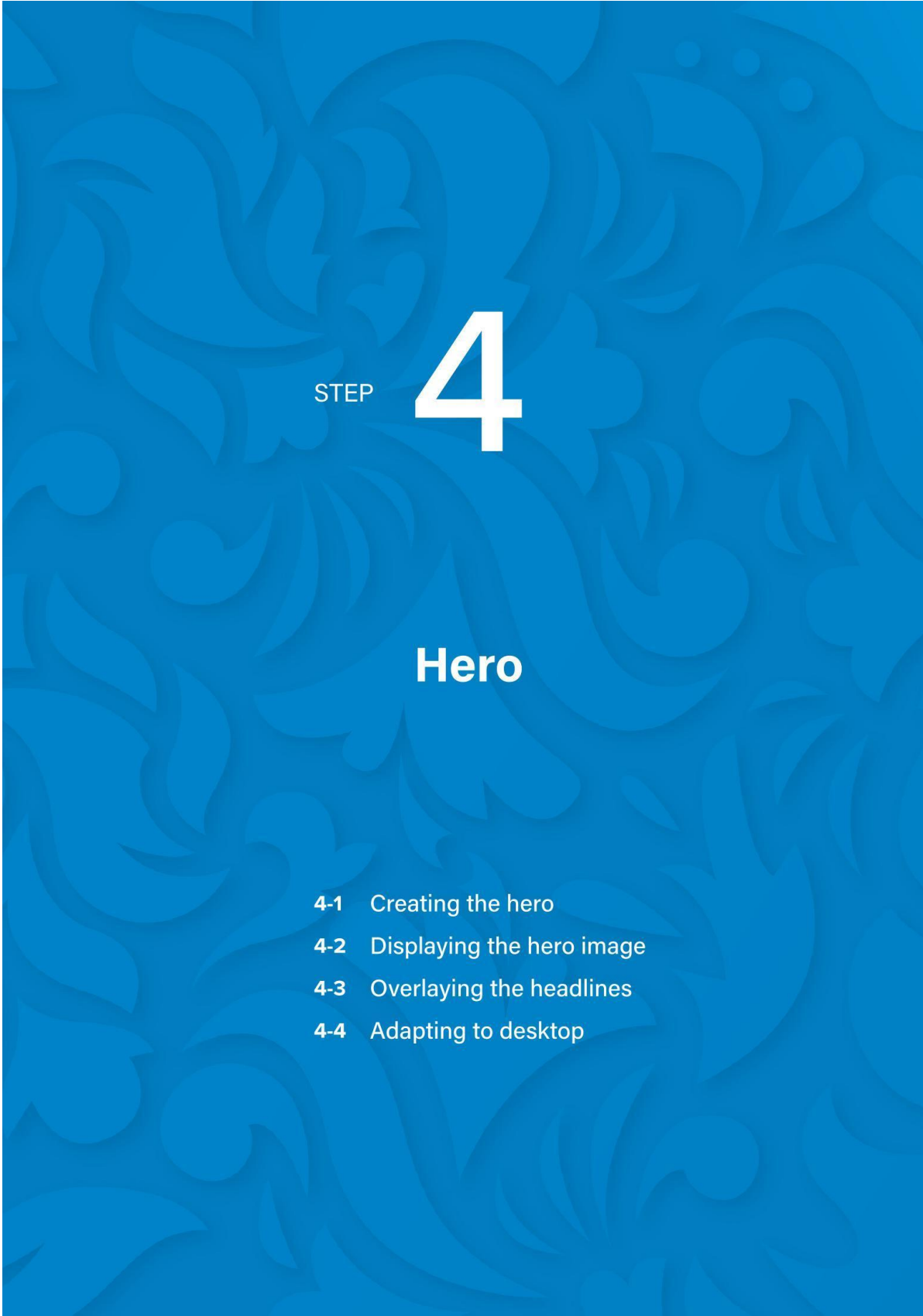
.site .desc {
  color: #707F89;
  font-size: 10px;
  line-height: 1.4;
}

@media(min-width: 900px) {
  .site img {
    width: 59px;
    margin-right: 18px;
  }

  .site .name {
    font-size: 22px;
  }

  .site .desc {
    font-size: 12px;
  }
}
```

style.css



STEP 4

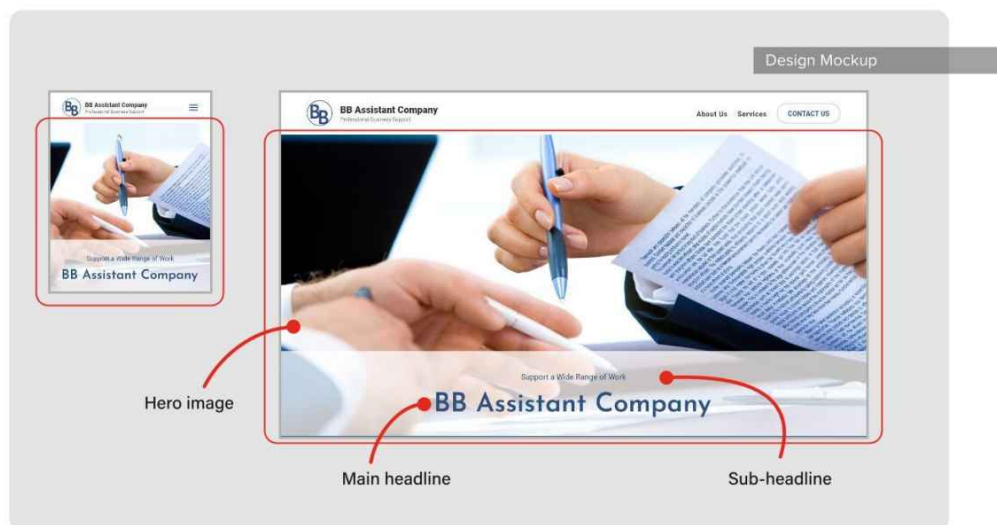
Hero

- 4-1 Creating the hero
- 4-2 Displaying the hero image
- 4-3 Overlaying the headlines
- 4-4 Adapting to desktop

4-1

Creating the hero

This step creates the ② hero part designed as an image with transparent background text (main and sub-headline).



Creation Step

We create the hero in the following steps.



Display the hero image.



Place headlines over the image.



Adjust headlines display order and placement to finish up.

4-2

Displaying the hero image

As determined in STEP 1-5, we create the hero with `<section class="hero">`. To make it easier to overlay the headlines, show the hero image (hero.jpg) as this `<section>` background.

Also, we set the `<section>` height to 400px as the image height. The image is clipped to fit the `<section>` size by specifying **background-position** and **background-size**.

```
...  
</header>  
  
<section class="hero">  
</section>  
  
</body>
```

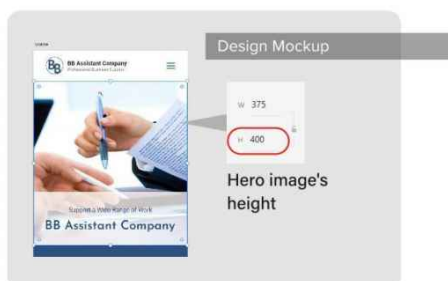
index.html

```
...  
.nav-button::before {  
  transform: translateY(-5px);  
  box-shadow: 0 6px currentColor;  
}  
  
/* Hero */  
.hero {  
  height: 400px;  
  background-image: url(img/hero.jpg);  
  background-position: center;  
  background-size: cover;  
}
```

style.css



The clipped hero image is displayed.



hero.jpg

4-3

Overlaying the headlines

We are going to place the headlines over the hero image.

First, add a main headline in `<section>` and define it as a top-level heading with `<h1>`. The mockup shows it as an SVG image (hero.svg), so use `<img>` to display and set alt text to 'BB Assistant Company.'

To make the image responsive, set **max-width** to 100% and **height** to auto. The **vertical-align** removes extra white space below the image.

Next, add sub-headline with `<p>`. The mockup shows a sub-headline placed above the main headline. But visually, the large main headline catches the eye first, followed by the smaller sub-headline.

So, we write the main headline `<h1>` before sub-headline `<p>` in the code.

```
<section class="hero">
  <h1>
    
    </h1>
    <p>Support a Wide Range of Work</p>
</section>
```

index.html

You could also write the sub-headline first. It depends on how the author wants to convey the content.



The main headline and sub-headline are displayed.

```
/* Hero */
.hero {
  height: 400px;
  background-image: url(img/hero.jpg);
  background-position: center;
  background-size: cover;
}

.hero img {
  max-width: 100%;
  height: auto;
  vertical-align: bottom;
}
```

style.css

Adjusting the headline style and alignment

The headline background is transparent (see-through) white, and its alignment is at the bottom of the hero image.

To achieve them, group `<h1>` and `<p>` with `<div class="headline">` whose background is set to transparent white by `rgba()`.

Then, add 33px **padding** to keep the top and bottom spaces in `<div class="headline">` and Flexbox's **align-self: flex-end** to make it aligned to the bottom of the hero image.

The content width of `<div class="headline">` is set to the same width (84%) as header content by adding `<div class="container">`.



We have adjusted the headline background and alignment.



```
<section class="hero">
  <div class="headline">
    <div class="container">
      <h1>
        
      </h1>
      <p>Support a Wide Range of Work</p>
    </div>
  </div>
</section>
```

index.html

```
/* Hero */
.hero {
  height: 400px;
  background-image: url(img/hero.jpg);
  background-position: center;
  background-size: cover;
  display: flex;
}

.hero .headline {
  flex: auto;
  align-self: flex-end;
  padding: 33px 0;
  background-color: rgba(255,255,255,0.76);
}

.hero img {
  max-width: 100%;
  height: auto;
  vertical-align: bottom;
}
```

style.css

Reversing the display order of headlines

We reverse the display order of the main headline `<h1>` and sub-headline `<p>` by using Flexbox. To do it, apply **display: flex** and **flex-direction: column-reverse** to the closest parent `<div class="container">` of `<h1>` and `<p>`. The **align-items** make them centered horizontally.

For sub-headline `<p>`, specify the font size and color. The **margin-bottom** adjusts the space between `<p>` and `<h1>`.

```
/* Hero */
.hero {
  height: 400px;
  background-image: url(img/hero.jpg);
  background-position: center;
  background-size: cover;
  display: flex;
}

.hero .headline {
  flex: auto;
  align-self: flex-end;
  padding: 33px 0;
  background-color: rgba(255,255,255,0.76);
}

.hero .container {
  display: flex;
  flex-direction: column-reverse;
  align-items: center;
}

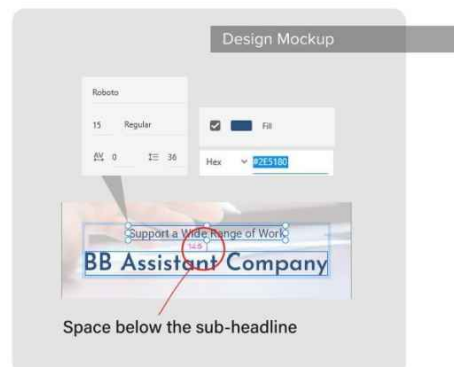
.hero img {
  max-width: 100%;
  height: auto;
  vertical-align: bottom;
}

.hero p {
  font-size: 15px;
  color: #2E5180;
  margin-bottom: 15px;
}
```

style.css



The sub-headline appears above the main headline.



Space below the sub-headline

4-4

Adapting to desktop

We are going to adjust the hero to adapt to the desktop screen. As the screen width increases, the main headline image (hero.svg) grows with it while preserving its aspect ratio.

However, it will not be larger than the original image width, 637.78px. It's the same width as the desktop version of the mockup, so we don't need to change the main headline image settings.



The main headline image becomes larger.

You'll see the image (hero.svg) width is 637.78px by selecting it in the browser's developer tools.



You'll see the main headline width is 637.78px by selecting it in the desktop version of the mockup.



```
.hero p {
  font-size: 15px;
  color: #2E5180;
  margin-bottom: 15px;
}

@media (min-width: 900px) {
  .hero {
    height: 700px;
  }

  .hero .headline {
    padding: 47px 0 44px;
  }

  .hero p {
    font-size: 18px;
    margin-bottom: 22px;
  }
}
```

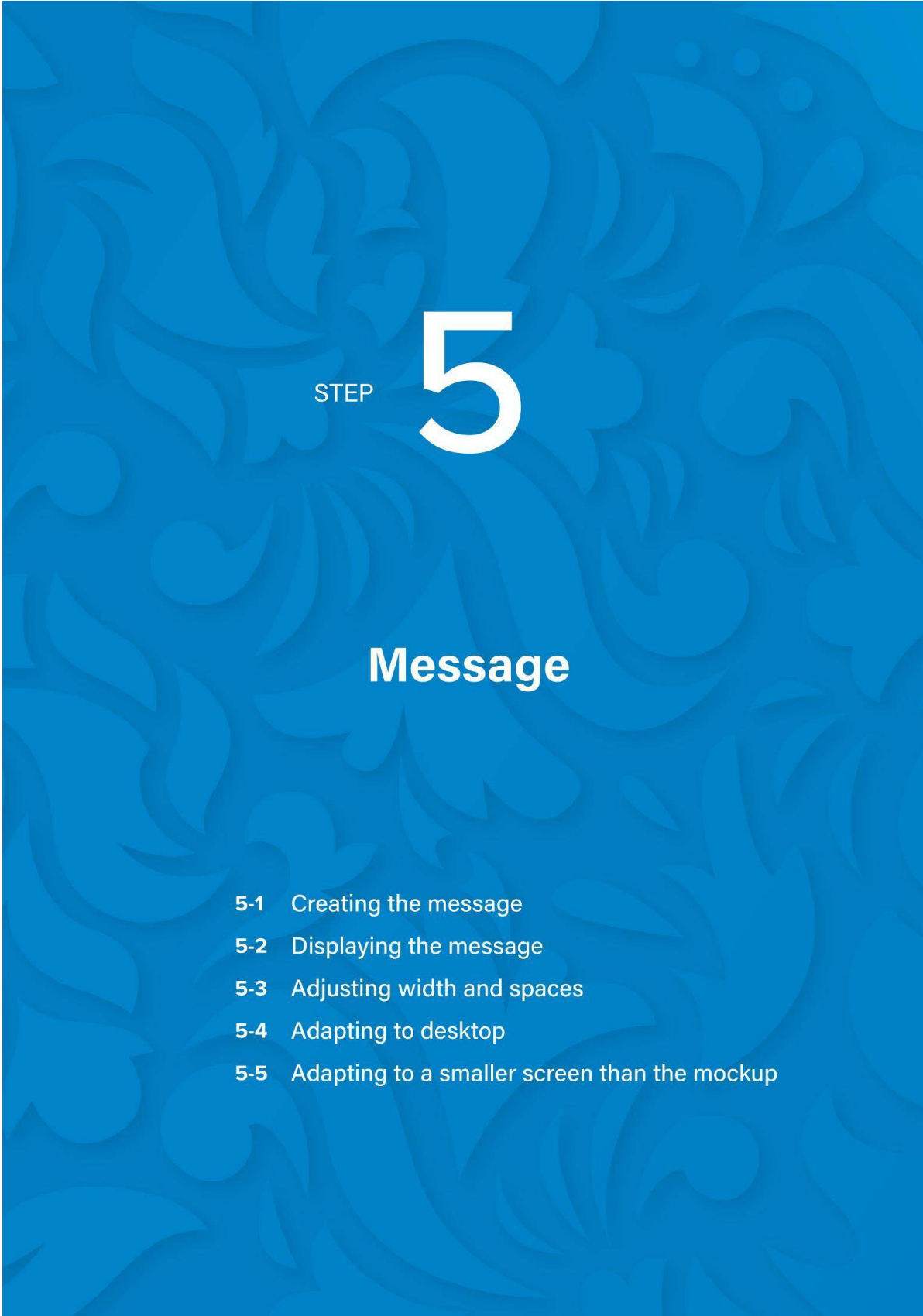
Top and bottom spaces

Height of the hero image

Space below the sub-headline

A font size of the sub-headline

Design Mockup



STEP 5

Message

- 5-1 Creating the message
- 5-2 Displaying the message
- 5-3 Adjusting width and spaces
- 5-4 Adapting to desktop
- 5-5 Adapting to a smaller screen than the mockup

5-1

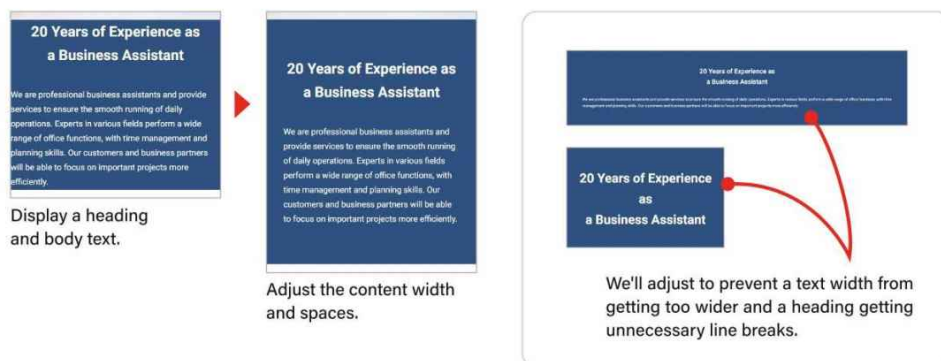
Creating the message

In this step, we create the ③ message part. It's a simple one with a heading and body text.



Creation Step

The creation step is also simple, but we need to fine-tune it to adapt to various screen sizes.



5-2

Displaying the message

As determined in STEP 1-5, we make the message with `<section class="message">`.

In this `<section>`, add a heading with `<h2>` and a body text with `<p>`. Then, set their styles, such as font size and color.

```

</section>
<section class="message">
  <h2>20 Years of Experience as<br>
  a Business Assistant</h2>
  <p>We are professional business assistants and
  provide services to ensure the smooth running
  of daily operations. Experts in various fields
  perform a wide range of office functions,
  with time management and planning skills. Our
  customers and business partners will be able to
  focus on important projects more efficiently.</p>
</section>
</body>

```

index.html

```

.hero p {
  font-size: 18px;
  margin-bottom: 22px;
}

/* Message */
.message {
  background-color: #2E5180;
  color: #ffffff;
}

.message h2 {
  margin-bottom: 38px;
  font-size: 26px;
  line-height: 1.62;
  text-align: center;
}

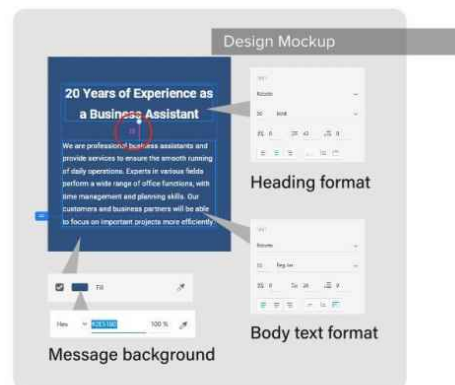
.message p {
  font-size: 15px;
  line-height: 1.73;
}

```

style.css



The message is displayed.



Set the message's background and text color.

Set the heading's space below, font size, line height, and text alignment. Although we have not specified font-weight, it gets bold by `<h2>`.

Set the body text's font size and line height.

The line-height value is calculated by $\text{XD's Line Spacing} \div \text{Font size}$. One for the heading is $42 \div 26 \approx 1.62$, for the body text is $26 \div 15 \approx 1.73$.



5-3

Adjusting width and spaces

To make the message content the same width as header content, we add `<div class="container">`.

The padding for `<section class="message">` adjust the top and bottom spaces in the message.

```
<section class="message">
  <div class="container">
    <h2>20 Years of Experience as<br>
    a Business Assistant</h2>
    <p>We are professional business assistants
    and provide services to ensure the smooth
    running of daily operations. Experts in
    various fields perform a wide range of
    office functions, with time management and
    planning skills. Our customers and business
    partners will be able to focus on important
    projects more efficiently.</p>
  </div>
</section>
```

index.html

```
/* Message */
.message {
  padding: 67px 0;
  background-color: #2E5180;
  color: #ffffff;
}
```

style.css



Adjustment result.



5-4

Adapting to desktop

As the screen width increases, the message body text gets wider than the mockup setting (640px width).

To prevent this, we set the max-width for `<p>` to 640px. Adding this setting outside `@media` makes it work on any screen width, not just the desktop.

For other settings that adjust spaces in the message and format of the heading, add them inside `@media` to apply if the screen (viewport) width is 900px or more.

Note that 900px is a temporary breakpoint defined in STEP 3-5.



```

.message p {
  max-width: 640px;
  margin: auto;
  font-size: 15px;
  line-height: 1.73;
}

@media (min-width: 900px) {
  .message {
    padding: 81px 0;
  }

  .message h2 {
    font-size: 36px;
    line-height: 1.33;
  }
}

```

style.css



Design Mockup

TEXT

Roboto

36 Bold

AV 0 0 48 0 0

Heading format

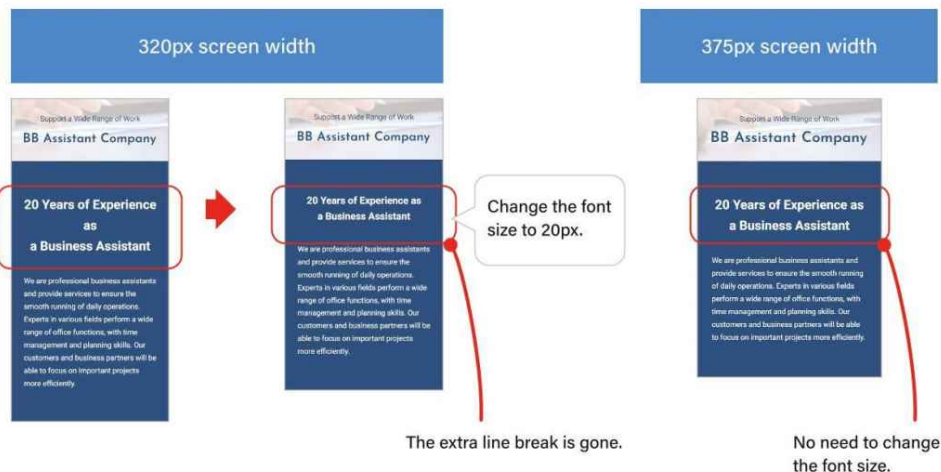
5-5

Adapting to a smaller screen than the mockup

The heading gets unnecessary line breaks as the screen width decreases than the mobile version mockup (375px width) — for example, the following is on the smallest screen (320px width) of major devices.

To prevent this, we decrease the heading font size if the screen (viewport) width is 374px or less.

That's all for the message part setting.



```

/* Message */
.message {
  padding: 67px 0;
  background-color: #2E5180;
  color: #ffffff;
}

.message h2 {
  margin-bottom: 38px;
  font-size: 26px;
  line-height: 1.62;
  text-align: center;
}

.message p {
  max-width: 640px;
  margin: auto;
  font-size: 15px;
  line-height: 1.73;
}

@media (min-width: 900px) {
  .message {
    padding: 81px 0;
  }
}

.message h2 {
  font-size: 36px;
  margin: auto;
  line-height: 1.33;
}

@media (max-width: 374px) {
  .message h2 {
    font-size: 20px;
  }
}

```

style.css

STEP 6

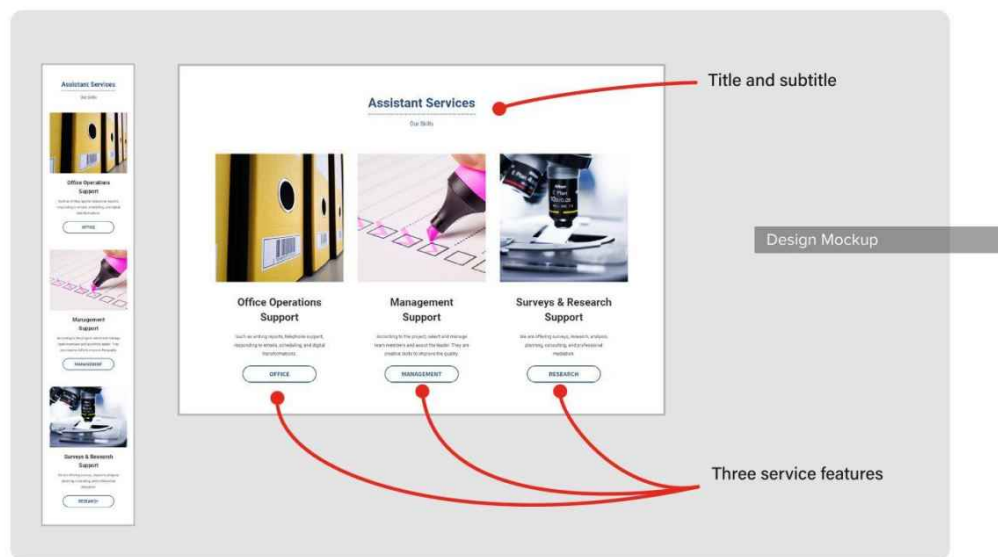
Service

- 6-1 Creating the service
- 6-2 Adding the title and subtitle
- 6-3 Adding the service features
- 6-4 Displaying the feature image
- 6-5 Styling the button
- 6-6 Aligning the features horizontally
- 6-7 Aligning button to the bottom

6-1

Creating the service

In this step, we create the ④ service part. This part includes the title, subtitle, and three service features.



Creation Step

This part has many elements, so we add and style them one by one. For the desktop layout, we'll tweak it at a temporary breakpoint (900px width), not only the mockup view (1366px width).



6-2

Adding the title and subtitle

As determined in STEP 1-5, we make the service part with `<section class="service">`. In this `<section>`, add title with `<h2>` and subtitle with `<p>`.

By the mockup, the service content has the same width as header content with top and bottom spaces. To achieve them, group the content `<h2>` and `<p>` with `<div class="container">`, plus insert spaces with top and bottom **padding**.

```
...
</section>

<section class="service">
  <div class="container">
    <h2>Assistant Services</h2>
    <p>Our Skills</p>
  </div>
</section>

</body>
```

index.html

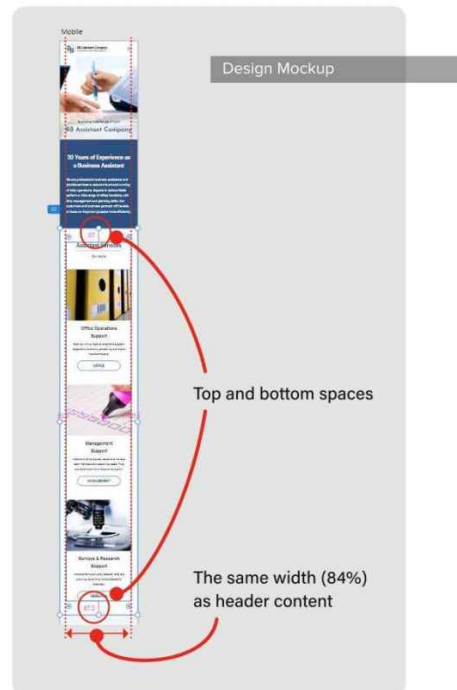
```
...
@media (max-width: 374px) {
  .message h2 {
    font-size: 20px;
  }
}

/* Service */
.service {
  padding: 67px 0;
}
```

style.css



The service title and subtitle are displayed.



Adjusting styles for the title and subtitle

For the title and subtitle, set text color to blue and align center. We apply **text-align: center** to `<section class="service">` so not only the titles but also the three features to be added later will be centered.

For a dotted border, we add it with the **border-bottom** of `<h2>` and apply **display: inline** to make its width fit the title.

For the desktop, adjust top and bottom spaces and font size.



Styling result.



```
...
/* Service */
.service {
  padding: 67px 0;
  text-align: center;
}

.service h2 {
  display: inline-block;
  margin-bottom: 16px;
  padding-bottom: 9px;
  border-bottom: dotted 2px currentColor;
  font-size: 26px;
  color: #2E5180;
}

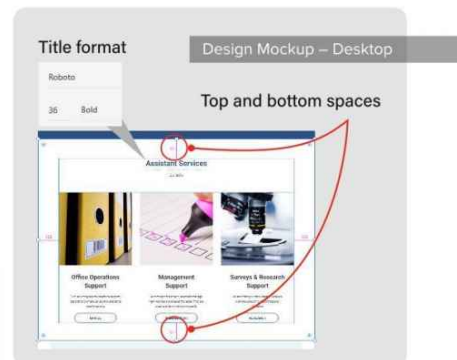
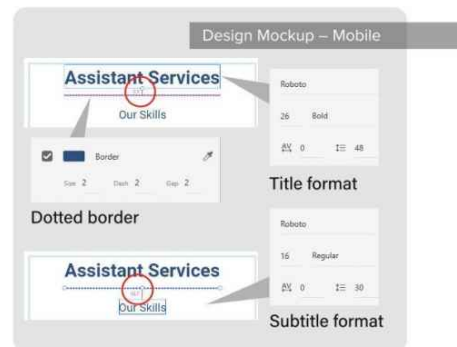
.service h2 + p {
  color: #2E5180;
  font-size: 16px;
}

@media (min-width: 900px) {
  .service {
    padding: 95px 0 87px;
  }

  .service h2 {
    font-size: 36px;
  }
}
```

style.css

The three features to be added later will also use `<p>`. So we use **.service h2 + p** selector to apply subtitle's setting only to `<p>` that is placed immediately follows `<h2>`.



6-3

Adding the service features

We add three service features into `<div class="container">`. For each feature, write heading with `<h3>`, body text with `<p>`, button with `<a>`, and group them with `<article>` to define as a self-contained content. We'll add image in STEP 6-4.

`<article>` has an individual class name 'office', 'manage', 'research' to identify each, and a common class name 'feature.' Also, to align them horizontally on desktop, make group with `<div class="features">`.

```
<section class="service">
  <div class="container">
    <h2>Assistant Services</h2>
    <p>Our Skills</p>

    <div class="features">
      <article class="feature office">
        <h3>Office Operations<br>Support</h3>
        <p>Such as writing reports, telephone support, responding to emails, scheduling, and digital transformations.</p>
        <a href="#">OFFICE</a>
      </article>

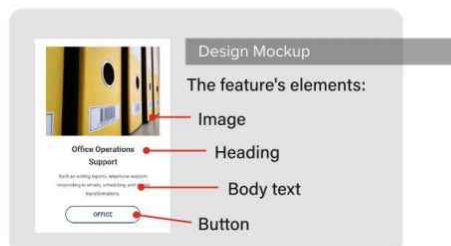
      <article class="feature manage">
        <h3>Management<br>Support</h3>
        <p>According to the project, select and manage team members and assist the leader. They are creative skills to improve the quality.</p>
        <a href="#">MANAGEMENT</a>
      </article>

      <article class="feature research">
        <h3>Surveys & Research<br>Support</h3>
        <p>We are offering surveys, research, analysis, planning, consulting, and professional mediation.</p>
        <a href="#">RESEARCH</a>
      </article>
    </div>
  </div>
</section>
```

index.html



Three service features are displayed.



Styling the features

For styling the features, adjust the formats such as font size and line height for the heading `<h3>` and body text `<p>`. For vertical spacing between elements, adjust with top and bottom margins. Also, limit the body text width with **max-width** and place it in the center with left and right margins set to 'auto.'

```

...
.service h2 {
  font-size: 36px;
}

/* Service Features */
.features {
  margin-top: 53px;
  color: #333333;
}

.feature:not(:last-child) {
  margin-bottom: 67px;
}

.feature h3 {
  margin-bottom: 17px;
  font-size: 22px;
  line-height: 1.6;
}

.feature p {
  max-width: 264px;
  margin: 0 auto 25px;
  font-size: 13px;
  line-height: 1.85;
}

```

Insert space between subtitle and features. Also, we set the feature's text color.

Set the spacing between features.

Set the heading's bottom margin, font size, and line height.

Set the body text's maximum width, margin, font size, and line height.

style.css

Assistant Services

Our Skills

Office Operations Support

Such as writing reports, telephone support, responding to emails, scheduling, and digital transformations.

OFFICE

Management Support

According to the project, select and manage team members and assist the leader. They are creative skills to improve the quality.

MANAGEMENT

Surveys & Research Support

We are offering surveys, research, analysis, planning, consulting, and professional mediation.

RESEARCH

Styling result.



6-4

Displaying the feature image

We add the feature image with `<img>` and define it with `<figure>` as an independent item. To make the image fluid, set its width to 100%.

```
<div class="features">
  <article class="feature office">
    <figure>
      
    </figure>
    <h3>Office Operations<br> Support</h3>
    ...
  </article>

  <article class="feature manage">
    <figure>
      
    </figure>
    <h3>Management<br> Support</h3>
    ...
  </article>

  <article class="feature research">
    <figure>
      
    </figure>
    <h3>Surveys & Research<br> Support</h3>
    ...
  </article>
</div>
```

index.html

```
...
.feature p {
  max-width: 264px;
  margin: 0 auto 25px;
  font-size: 13px;
  line-height: 1.85;
}

.feature figure img {
  width: 100%;
  vertical-align: bottom;
}
```

style.css

The specification doesn't provide alt text, so we set it as a decorative image with an empty value.

The vertical-align removes extra white space below the image.



The feature images are displayed.

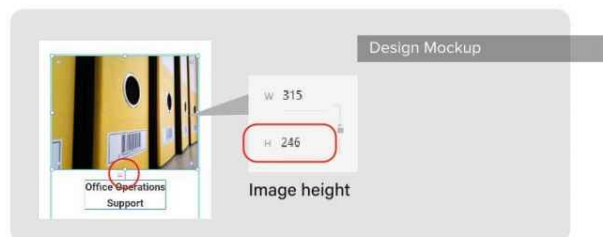
Adjusting the image height

We have displayed the image, but its height is smaller than the one in the mockup.

To show the same as the mockup, set the **height** to 246px and **object-fit** to 'cover.' It makes the image clipped to fit its specified size. The **margin-bottom** inserts a space below the image.

```
...  
.feature figure img {  
  width: 100%;  
  height: 246px;  
  object-fit: cover;  
  margin-bottom: 22px;  
  vertical-align: bottom;  
}
```

style.css



The results show differences depending on whether using **width** or **max-width** to set the image width 100%. Each result looks like below as the screen width increases.

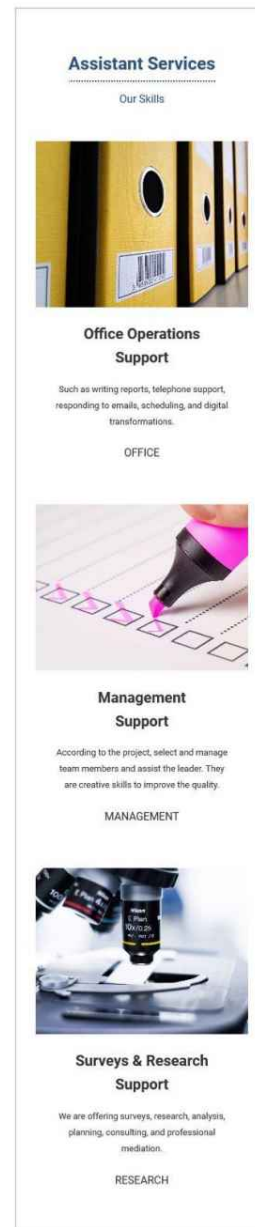
We use the **width** since the specification doesn't define how to display the image between mockup and desktop.



width: 100%
It changes the image width along the screen width.



max-width: 100%
The specified height determines the maximum image width.



The image height gets larger.

6-5

Styling the button

We add a class name 'btn' to <a> for button styling. According to STEP 1-6, make its CSS available throughout the page.

```
<div class="features">
  <article class="feature office">
    ...
    <a href="#" class="btn">OFFICE</a>
  </article>

  <article class="feature manage">
    ...
    <a href="#" class="btn">MANAGEMENT</a>
  </article>

  <article class="feature research">
    ...
    <a href="#" class="btn">RESEARCH</a>
  </article>
</div>
```

index.html

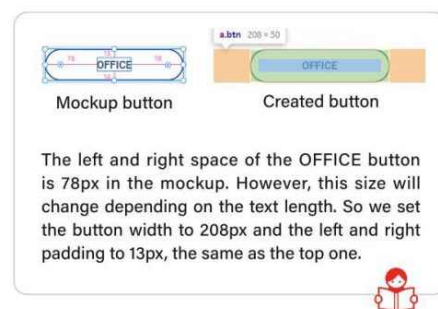
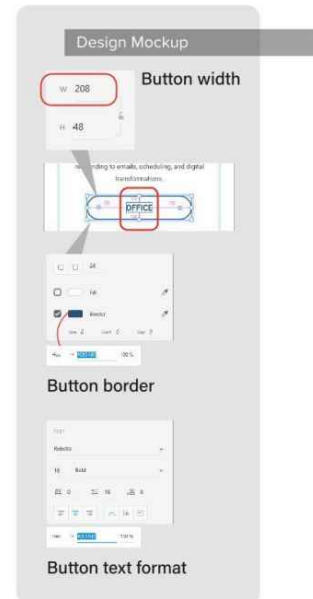
```
...
.feature figure img {
  width: 100%;
  height: 246px;
  object-fit: cover;
  margin-bottom: 22px;
  vertical-align: bottom;
}

/* Button */
.btn {
  display: block;
  width: 208px;
  margin: 0 auto;
  padding: 13px 13px 14px;
  box-sizing: border-box;
  border: solid 2px currentColor;
  border-radius: 24px;
  color: #2E5180;
  font-size: 16px;
  font-weight: bold;
  text-align: center;
}
```

style.css



Styling results



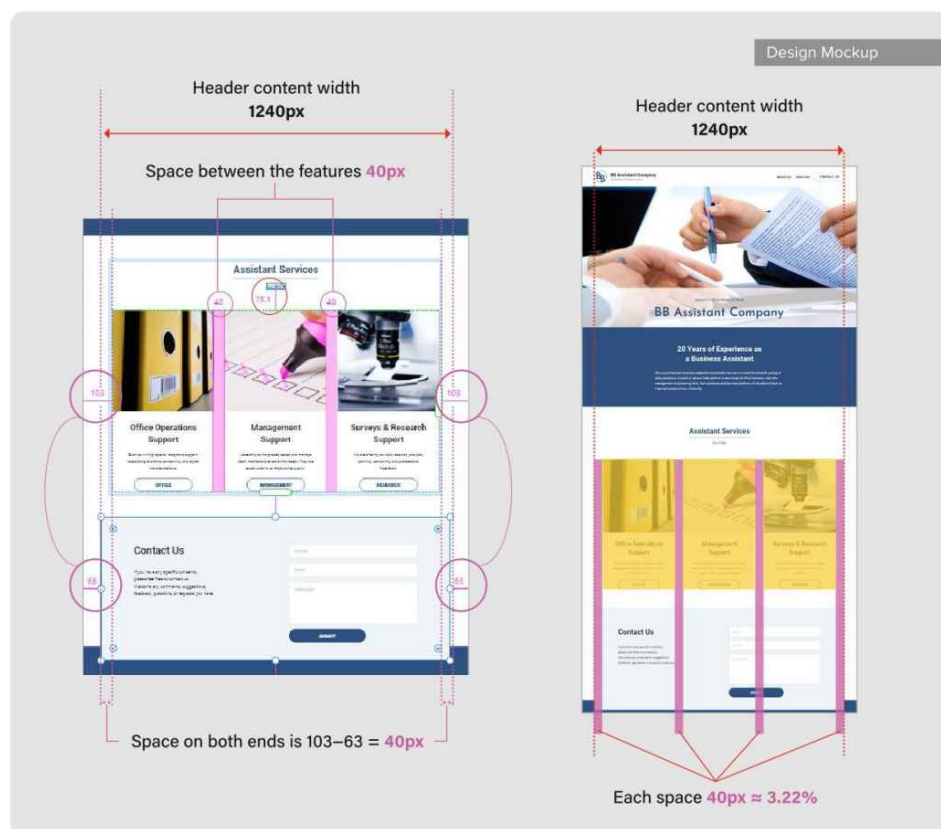
6-6

Aligning the features horizontally

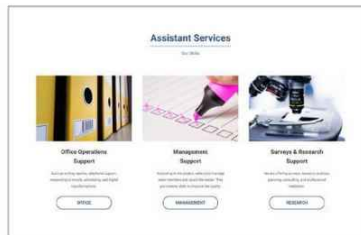
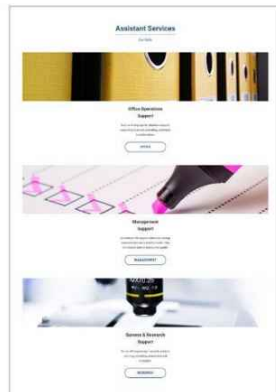
The desktop version of the mockup shows:

- ✓ The three features are aligned side by side horizontally.
- ✓ The three features' width is a little smaller than the header content width (1240px).
- ✓ Each space on the left, right, and between the features is 40px.

To make it fluid, specify the space size as a percentage. The header content width is 1240px, so 40px is $40 \div 1240 \times 100 \approx 3.22\%$.



To align three features horizontally, we apply **display: flex** to `<div class="features">`. Then, set the **margin-right** of all `<article class="feature">` and **margin-left** of first one to 3.22% (40px). Each feature needs to have the same width, so apply **flex: 1** to `<article class="feature">` (which makes its width 1/3 of available free space except margin).



The three features are aligned horizontally.

```
/* Service Features */
...
margin-bottom: 22px;
vertical-align: bottom;
}

@media (min-width: 900px) {
  .features {
    display: flex;
    margin-top: 75px;
  }

  .feature {
    flex: 1;
    margin-right: 3.22%;
  }

  .feature:first-child {
    margin-left: 3.22%;
  }

  .feature:not(:last-child) {
    margin-bottom: 0;
  }
}

/* Button */
```

style.css

Delete the bottom margin of `<article>` inserted for the mobile layout.



Adjusting the styles

We adjust the styles for the desktop layout, such as font size and line height for heading, width for body text, height for image, and margins under image and heading.

```
.feature h3 {
  margin-bottom: 17px;
  font-size: 22px;
  line-height: 1.6;
}

.feature p {
  max-width: 264px;
  margin: 0 auto 25px;
  font-size: 13px;
  line-height: 1.85;
}

.feature figure img {
  width: 100%;
  height: 246px;
  object-fit: cover;
  margin-bottom: 22px;
  vertical-align: bottom;
}

@media (min-width: 900px) {
  ...
  .feature:not(:last-child) {
    margin-bottom: 0;
  }

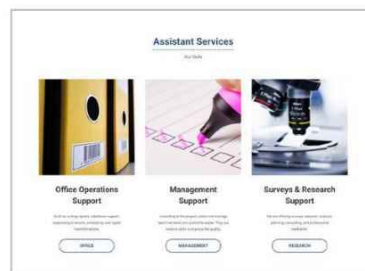
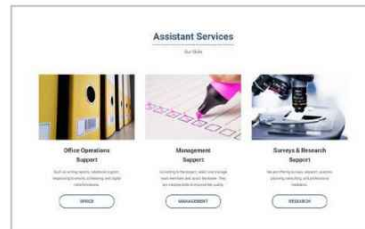
  .feature h3 {
    margin-bottom: 26px;
    font-size: 30px;
    line-height: 1.4;
  }

  .feature p {
    width: 78%;
    max-width: none;
  }

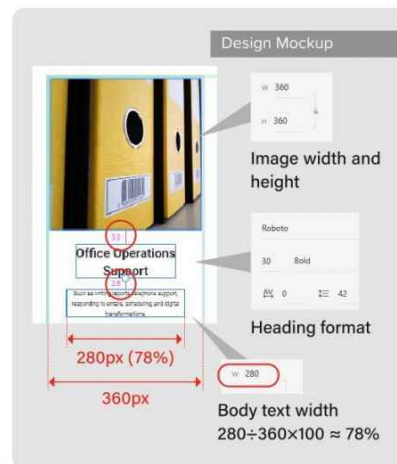
  .feature figure img {
    height: 360px;
    margin-bottom: 33px;
  }
}

/* Button */
```

style.css



Adjustment result.



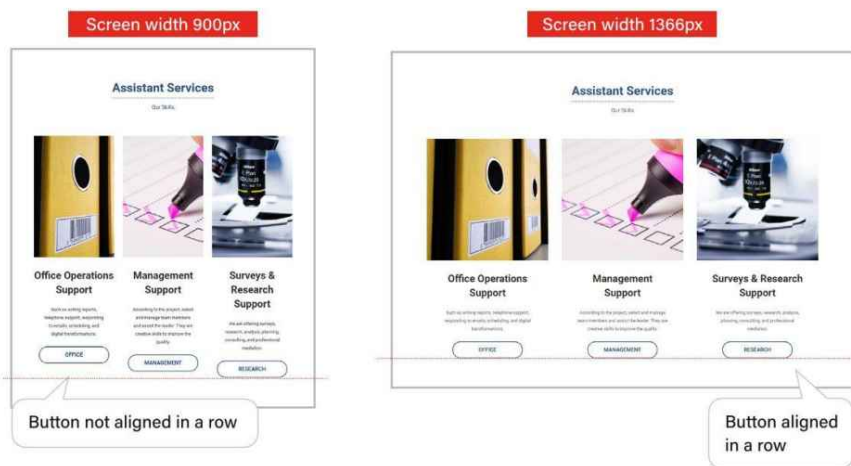
To make it fluid while keeping the balance in layout, specify the body text `<p>` width 280px as 78% of the parent `<article>` width 360px.



6-7

Aligning button to the bottom

As the screen width decreases, getting the feature's width narrower causes buttons not aligned in a row with each other.



To fix the above problem, use the flexbox and align button to the bottom of the feature. Now the service part is complete.

```

@media (min-width: 900px) {
  ...
  .feature figure img {
    height: 360px;
    margin-bottom: 33px;
  }

  /* Button Alignment */
  .feature {
    display: flex;
    flex-direction: column;
  }

  .feature .btn {
    margin-top: auto;
  }
}

/* Button */
        
```

To align the button to the bottom, make the feature's content (image, heading, body text, button) stacked vertically by the Flexbox and insert auto margin above the button.

How to align button to the bottom

The Flexbox makes the button aligned to the bottom as follows.



Assistant Services

Office Operations Support

Management Support

Surveys & Research Support

OFFER MANAGEMENT REQUEST



Assistant Services

Office Operations Support

Management Support

Surveys & Research Support

OFFER MANAGEMENT REQUEST

Extra space

The assumption is that the Flexbox of `<div class="features">` makes its child `<article class="feature">` not only aligned horizontally, but also set the same height.

As a result, there is **extra space** under the first and second `<a>` buttons.





Assistant Services

Office Operations Support

Management Support

Surveys & Research Support

OFFER MANAGEMENT REQUEST



Assistant Services

Office Operations Support

Management Support

Surveys & Research Support

OFFER MANAGEMENT REQUEST

Then, set the Flexbox of `<article class="feature">` to stack its child `<figure>`, `<h3>`, `<p>`, `<a>` vertically.

The display will not change, but it's ready to control the **extra space**.

```
.feature {
  display: flex;
  flex-direction: column;
}
```





Assistant Services

Office Operations Support

Management Support

Surveys & Research Support

OFFER MANAGEMENT REQUEST



Assistant Services

Office Operations Support

Management Support

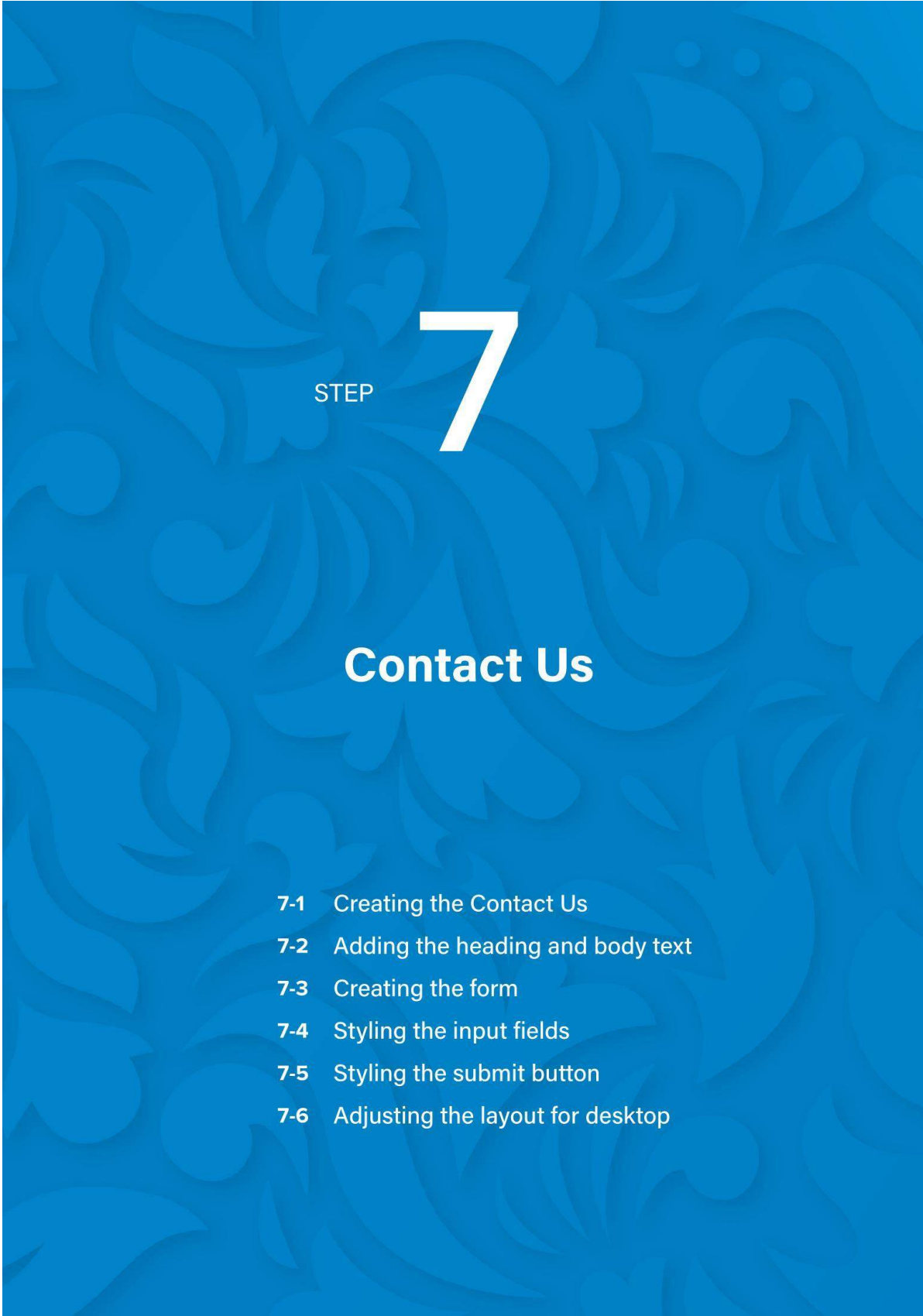
Surveys & Research Support

OFFER MANAGEMENT REQUEST

Set the **margin-top** of the button `<a>` as 'auto'.

It will automatically extend the margin to occupy the **extra space** and push the button into the parent's bottom.

```
.feature {
  display: flex;
  flex-direction: column;
}
.feature .btn {
  margin-top: auto;
}
```



STEP 7

Contact Us

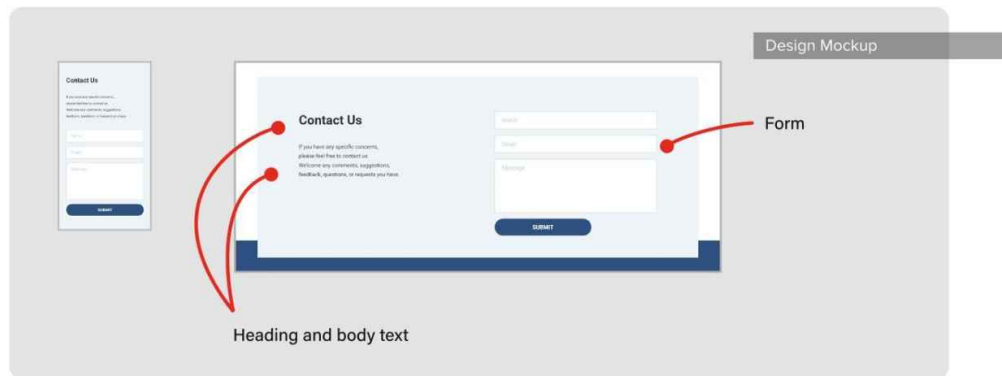
- 7-1 Creating the Contact Us
- 7-2 Adding the heading and body text
- 7-3 Creating the form
- 7-4 Styling the input fields
- 7-5 Styling the submit button
- 7-6 Adjusting the layout for desktop

7-1

Creating the Contact Us

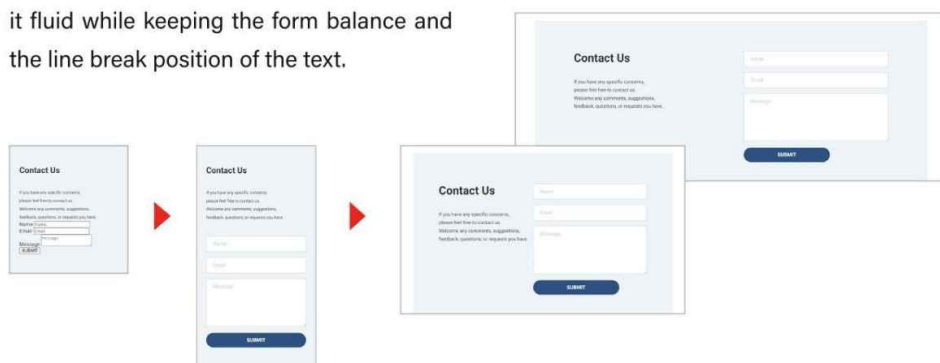
In this step, we create the 5 Contact Us part. This part includes the heading, body text, and form.

The mobile layout fills the screen width, but the desktop has the left and right white spaces. As for the form, each field has no label, only a placeholder.



Creation Step

We add heading, body text, form and adjust their styles. For the desktop layout, make it fluid while keeping the form balance and the line break position of the text.



7-2

Adding the heading and body text

As determined in STEP 1-5, we make the Contact Us part with `<section class="contact">`. In this `<section>`, add heading with `<h2>`, body text with `<p>`, and group them with `<div class="text">`. This makes it easier to align the text group next to the form in the desktop layout. Also, group them with `<div class="container">` to make the Contact Us content the same width as header content. Then insert top and bottom spaces with padding of `<section class="contact">`.

```

</section>

<section class="contact">
  <div class="container">
    <div class="text">
      <h2>Contact Us</h2>
      <p>If you have any specific concerns,<br>
      please feel free to contact us.<br>
      Welcome any comments, suggestions,<br>
      feedback, questions, or requests you have.</p>
    </div>
  </div>
</section>

</body>

```

index.html

```

/* Button */
.btn {
  ...
  text-align: center;
}

/* Contact Us */
.contact {
  padding: 62px 0;
  background-color: #EEF4F8;
}

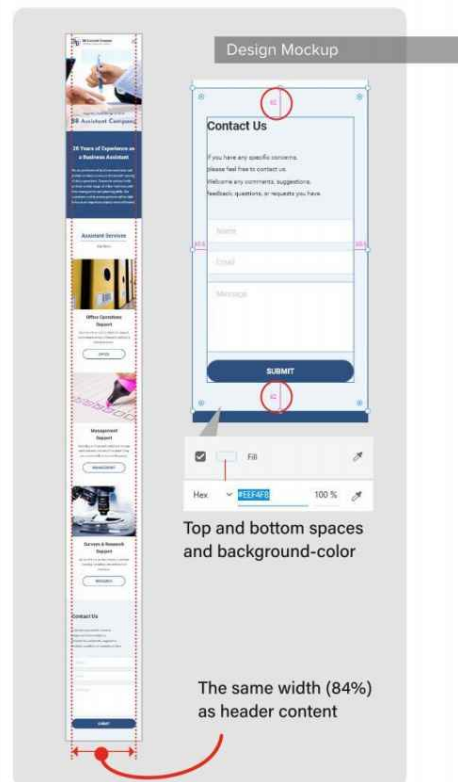
```

style.css

Contact Us

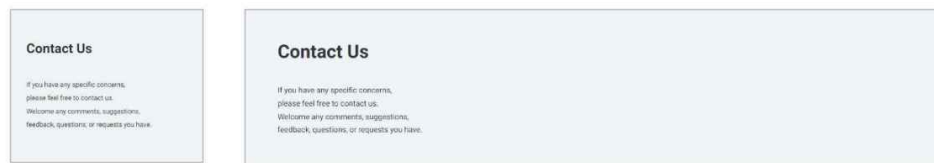
If you have any specific concerns,
please feel free to contact us.
Welcome any comments, suggestions,
feedback, questions, or requests you have.

Heading and body text are displayed.



Adjusting styles for the heading and body text

Set the heading and body text's spacing, color, font size, and line height for mobile and desktop. For the left and right white spaces in the desktop layout, we'll adjust later.



Adjustment result.

```
...
/* Contact Us */
.contact {
  padding: 62px 0;
  background-color: #EEF4F8;
  color: #333333;
}

.contact h2 {
  margin-bottom: 42px;
  font-size: 26px;
}

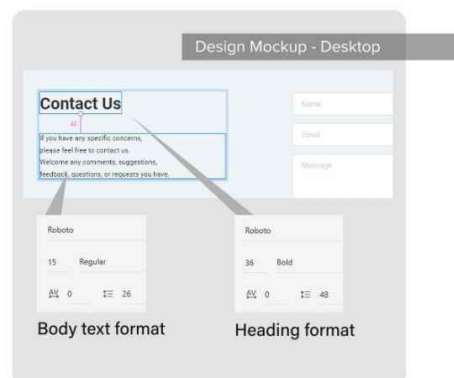
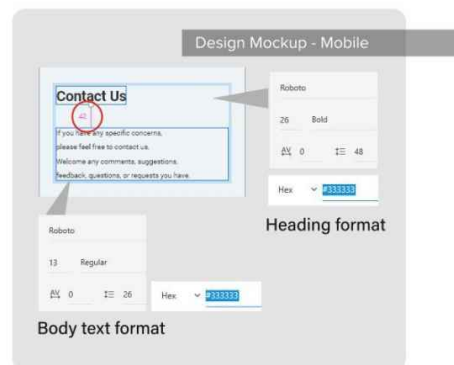
.contact h2 + p {
  font-size: 13px;
  line-height: 2;
}

@media (min-width: 900px) {
  .contact h2 {
    font-size: 36px;
  }

  .contact h2 + p {
    font-size: 15px;
    line-height: 1.73;
  }
}
```

style.css

We use **.contact h2 + p** selector to apply text's setting only to <p> that is placed immediately follows <h2>. This way, we can prevent applying the setting to another <p> that we'll add later.



7-3

Creating the form

We create the form in `<div class="container">`. The form has three input fields and a submit button. We labeled them 'Name,' 'Email,' and 'Message' with `<label>` to make it easier to understand each field.

Also, show the placeholder text in the field by specifying the placeholder attribute of `<input>`.

The form is displayed.

```
<section class="contact">
  <div class="container">
    <div class="text">
      <h2>Contact Us</h2>
      <p>If you have any specific concerns,<br>
      please feel free to contact us.<br>
      Welcome any comments, suggestions,<br>
      feedback, questions, or requests you have.</p>
    </div>

    <form class="form">
      <p>
        <label>
          Name
          <input type="text" name="name"
            placeholder="Name">
        </label>
      </p>
      <p>
        <label>
          Email
          <input type="email" name="email"
            placeholder="Email">
        </label>
      </p>
      <p>
        <label>
          Message
          <textarea name="msg"
            placeholder="Message"></textarea>
        </label>
      </p>
      <p>
        <input type="submit" value="SUBMIT">
      </p>
    </form>
  </div>
</section>
```

Add three fields with `<input>`:
Name, Email, Message.
`<p>` defines each field as one
paragraph, and `<form>` defines
the whole as one form.

Add the submit button with
`<input>`. The value attribute
specifies the button label.

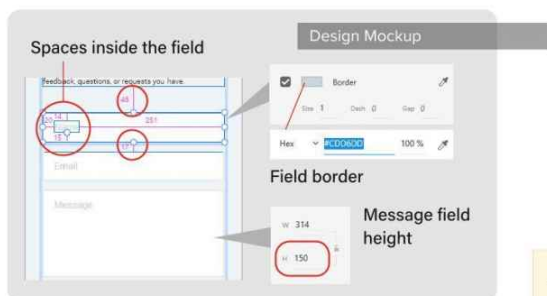
index.html

7-4

Styling the input fields

We style the input fields. First, add `<span class="sr-only">` to hide the label from display except screen readers.

Then, set the field width to 100% to make it full width. Adjust their spacing with margin and inside space with padding.



```

<form class="form">
  <p>
    <label>
      <span class="sr-only">Name</span>
      <input type="text" name="name"
        placeholder="Name">
    </label>
  </p>
  <p>
    <label>
      <span class="sr-only">Email</span>
      <input type="email" name="email"
        placeholder="Email">
    </label>
  </p>
  <p>
    <label>
      <span class="sr-only">Message</span>
      <textarea name="msg"
        placeholder="Message"></textarea>
    </label>
  </p>
</form>

```

index.html

```

...
    line-height: 1.73;
  }
}

/* Contact Us Form */
.form {
  margin-top: 46px;
}

.form input[type="text"],
.form input[type="email"],
.form textarea {
  width: 100%;
  margin-bottom: 17px;
  padding: 14px 20px 15px;
  border: solid 1px #CDD6DD;
  box-sizing: border-box;
  border-radius: 0;
  -webkit-appearance: none;
}

.form textarea {
  height: 150px;
}

```

style.css

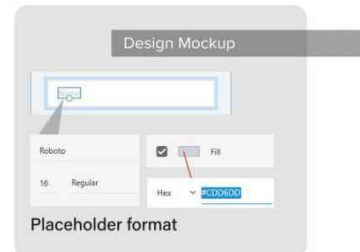
We set the right space (padding) to 20px, the same as the left.



Specifying the field text format

The field shows the placeholder or entered text with the browser's default format. To change their format, set the font family, size, color for `<input>` and `<textarea>`.

However, the mockup doesn't show the entered text format. So we set it to the same as the placeholder except for the color.



Placeholder

Entered text

In Chrome on Windows:
Arial, 13.3333px, #000000

In Chrome on Windows:
Consolas, 13.3333px, #000000

In macOS Safari (all fields):
system-ui, 11px, #000000

Display with Roboto, 16px, #333333

We set the entered text color to dark gray (#333333), the same as the heading and body text of the Contact Us part.

```
.form input[type="text"],
.form input[type="email"],
.form textarea {
  ...
  -webkit-appearance: none;
  color: #333333;
  font-family: 'Roboto', sans-serif;
  font-size: 16px;
}

.form ::placeholder {
  color: #CDD6DD;
}

.form textarea {
  height: 150px;
}
```

style.css

We use the **:placeholder** selector to apply this color (#CDD6DD) only to the placeholder.

7-5

Styling the submit button

We style the submit button based on the 'btn' setting created in STEP 6-4.

Adding the 'btn' class to `<input type="submit">` makes it the rounded corner button. All we have to do now is adjust the width, color, font, etc., and finish it up.

```
<p>
  <input type="submit" value="SUBMIT"
    class="btn">
</p>
```

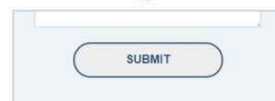
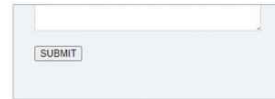
index.html

```
/* Button */
.btn {
  display: block;
  width: 208px;
  margin: 0 auto;
  padding: 11px 11px 13px;
  box-sizing: border-box;
  border: solid 2px currentColor;
  border-radius: 24px;
  color: #2E5180;
  font-size: 16px;
  font-weight: bold;
  text-align: center;
}

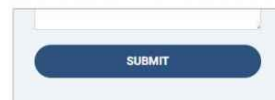
.form textarea {
  height: 150px;
}

.form .btn {
  width: 100%;
  margin: 0;
  border: none;
  background-color: #2E5180;
  color: #ffffff;
  font-family: 'Roboto', sans-serif;
  cursor: pointer;
  -webkit-appearance: none;
}
```

style.css



Display when the class 'btn' is specified.



Adjustment result.



The 'btn' setting.

Make the button width 100%, the same as the input field, and delete the margin and border.

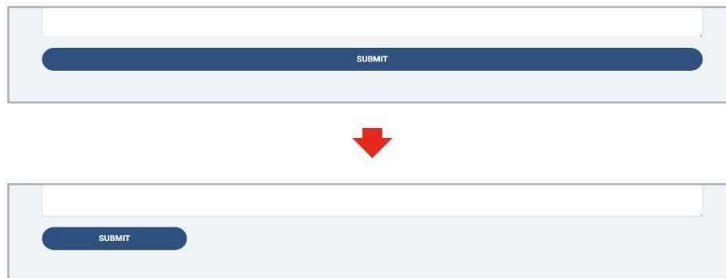
Set the background color to blue (#2E5180), the label color to white (#ffffff), and the font family to Roboto.

Set the hovering mouse cursor to the pointing hand. The -webkit-appearance resets the default <button> styles for iOS.



Adjusting the button style for the desktop

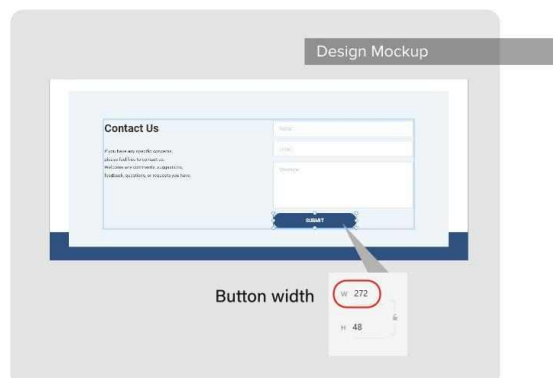
The submit button gets too wide as the screen width increases. So we set it to the fixed width of 272px for the desktop. Now the button adjustment is complete.



The button gets the fixed width.

```
.form .btn {  
  width: 100%;  
  margin: 0;  
  border: none;  
  background-color: #2E5180;  
  color: #ffffff;  
  cursor: pointer;  
  -webkit-appearance: none;  
}  
  
@media (min-width: 900px) {  
  .form .btn {  
    width: 272px;  
  }  
}
```

style.css



7-6

Adjusting the layout for desktop

We want to make the Contact Us part for the desktop the same width as the header content and insert the white space into the left and right sides. However, we would achieve them usually by adding `<div class="container">`, but it's already in use.

It's difficult to change the HTML markup in responsive, so we'll modify the appearance with CSS. In this case, we set the light gray background for `<div class="container">` and delete the one for `<section class="contact">`.

The same width as header content

The same width as header content

```
<section class="contact">
  <div class="container">
    <div class="text">
      <h2>Contact Us</h2>
      ...
    </div>

    <form class="form">
      ...
    </form>
  </div>
</section>
```

index.html

```
/* Contact Us */
.contact {
  padding: 62px 0;
  background-color: #EEF4F8;
  color: #333333;
}

@media (min-width: 900px) {
  .contact {
    background: none;
  }

  .contact .container {
    background-color: #EEF4F8;
  }

  .contact h2 {
```

style.css

Background color for `<section class="contact">`

Delete the background color for `<section class="contact">`

Set the background color for `<div class="container">`

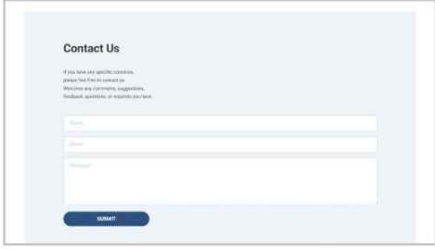
Next, to adjust the spaces, we delete all padding of `<section class="contact">` and add padding to `<div class="container">`. Now the Contact Us part appears to be the same width as the header content.



`<section class="contact">`

`<div class="container">`

The same width as header content



`<section class="contact">`

`<div class="container">`

The same width as header content

```

.container {
  width: 84%;
  margin: 0 auto;
}

@media (min-width: 900px) {
  .container {
    width: 91%;
    max-width: 1240px;
  }
}

@media (min-width: 900px) {
  .contact {
    padding: 0;
    background: none;
  }

  .contact .container {
    padding: 96px 9% 62px;
    box-sizing: border-box;
    background-color: #EEF4F8;
  }

  .contact h2 {
    font-size: 36px;
  }

```

style.css

Padding of `<div class="container">`

Left and right margin of `<div class="container">`

Design Mockup



The same width (1240px) as header content

To make it fluid while keeping the layout balance, specify the left and right padding 116px as 9% of the container width 1240px. The `box-sizing: border-box` makes the padding included in the width.

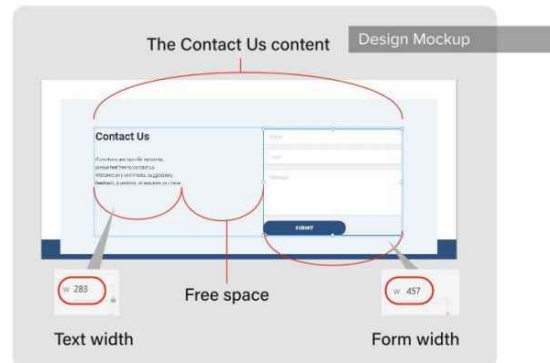
Planning the width for the text and form

We want to make the Contact Us content, text and form, align horizontally. In this case, planning their width will be helpful to make them responsive.

In the mockup, the width of the text is 283px, and the form is 457px. Plus, the free space is in between them.

If only the free space is fluid, aligning the text and form horizontally needs to be at least 740px width ($283 + 457 = 740$ px).

The 1366px mockup screen width is fine, but how about the 900px breakpoint?



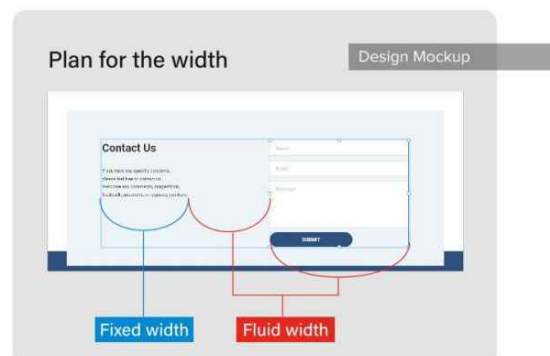
Available space for their alignment is only the Contact Us content width. It will be 657px at the 900px breakpoint. So the text and form also need to be fluid.



To check the content width at the breakpoint, use the developer tool on the browser.

However, we control the text line break with `<br>`. Making the text fluid may result in unwanted line breaks.

Therefore, we decided to make the text width **fixed** and the form and the free space **fluid**.



Aligning the text and form horizontally

We align the text and form horizontally with Flexbox. To do so, apply **display: flex** to `<div class="container">`. The **justify-content: space-between** inserts free space between them.

To make the text `<div class="text">` fixed width, use **flex: 0 0 auto** which disables growth and shrinks.

To make the form `<div class="form">` fluid width, use **flex: 1 1 auto** which enables growth and shrinks. However, limit the **max-width** to 457px.

The **margin-left** set the minimum space between the text and form to 1em (for one character). The **margin-top** removes the top margin inserted for the mobile layout.

Now the Contact Us part is complete.

```
/* Contact Us */
...
@media (min-width: 900px) {
  .contact {
    padding: 0;
    background: none;
  }

  .contact .container {
    display: flex;
    justify-content: space-between;
    padding: 96px 9% 62px;
    box-sizing: border-box;
    background-color: #EEF4F8;
  }

  .contact .text {
    flex: 0 0 auto;
  }

  .contact .form {
    flex: 1 1 auto;
    max-width: 457px;
    margin-left: 1em;
    margin-top: 0;
  }

  .contact h2 {
    font-size: 36px;
  }
}
```

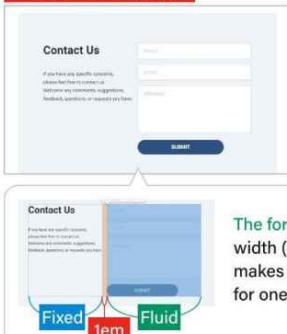
style.css

Screen width 1366px



The free space appears between the text and the form.

Screen width 900px



The form gets narrower than max-width (457px), and its left margin makes the minimal center space for one character (1em = 16px).

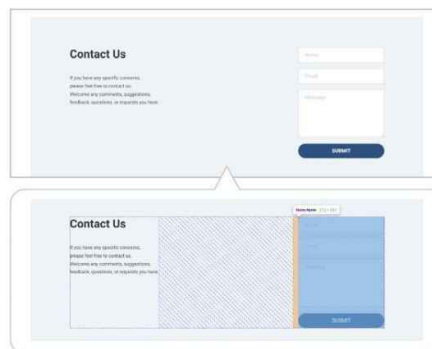
The form width

Adjusting width of the form `<form class="form">` makes the result as follows.

Case 1: make it fixed width

The form will be fixed width fit to its button by applying **flex: 0 0 auto** as same as the text.

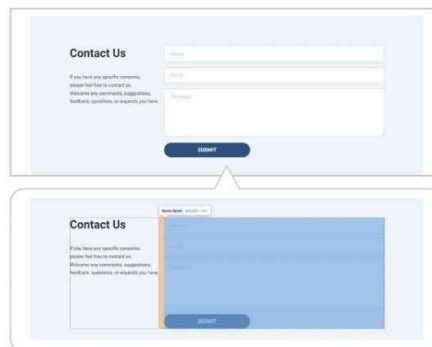
```
.contact .text {  
  flex: 0 0 auto;  
}  
  
.contact .form {  
  flex: 0 0 auto;  
  max-width: 457px;  
  margin-left: 1em;  
  margin-top: 0;  
}
```



Case 2: not specify the max-width

If not specified the max-width, the form will stretch to fill the available space except the text and 1em (for one character) margin.

```
.contact .text {  
  flex: 0 0 auto;  
}  
  
.contact .form {  
  flex: 1 1 auto;  
  max-width: 457px;  
  margin-left: 1em;  
  margin-top: 0;  
}
```





STEP 8

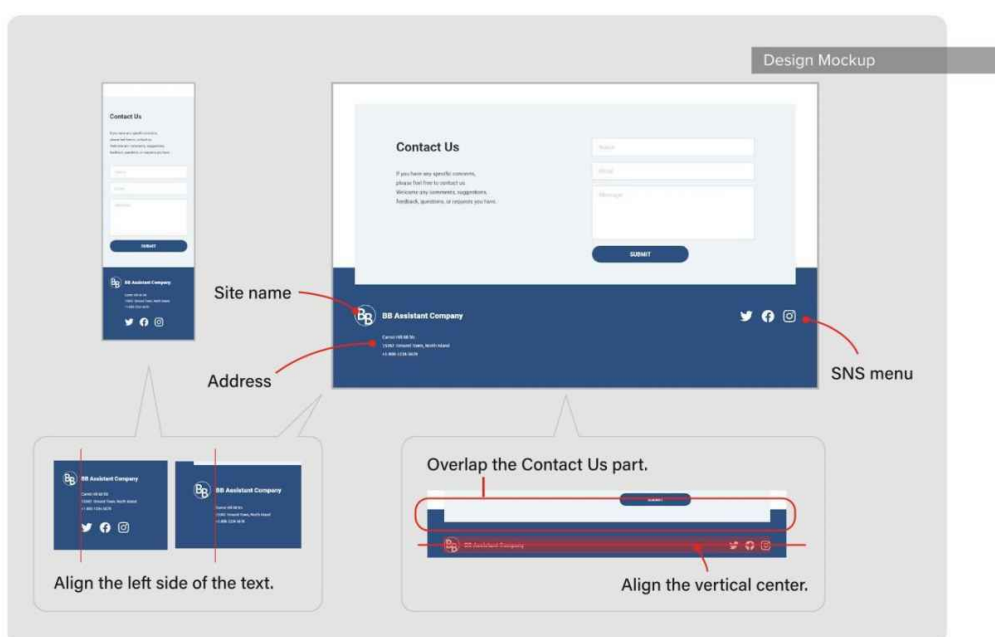
Footer

- 8-1 Creating the footer
- 8-2 Displaying the site name
- 8-3 Adding the address
- 8-4 Adding the SNS menu
- 8-5 Adjusting the layout for desktop

8-1

Creating the footer

The 6 footer part includes the site name, address, and SNS menu. We will create it with attention to the text alignment position and the overlapping of the Contact US part.



Creation Step

Adding and styling the footer content is done one by one. For the desktop layout, overlap it with the Contact Us part finally.

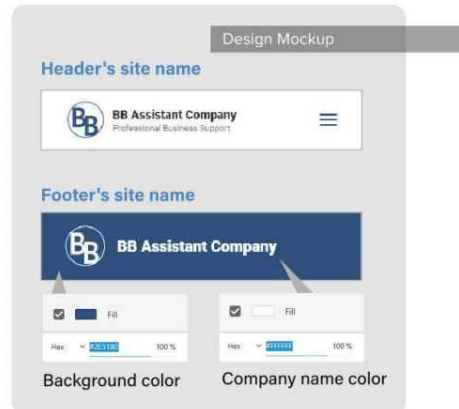


8-2

Displaying the site name

We make the footer part with `<footer class="footer">` as determined in STEP 1-5, applying a blue background.

The footer's site name has a logo image and company name. It's the same structure as the header's site name without the description. So we copy the header's one's `<div class="site">` and insert it with the description `<p class="desc">` removed. Additionally, replace the logo image with a white one (bb-white.svg) and set the company name color to white (#ffffff).



```
<header class="header">
  <div class="container">
    <div class="site">
      
      <div>
        <p class="name">BB Assistant Company</p>
        <p class="desc">
          Professional Business Support</p>
      </div>
    </div>
  </div>
</header>

...

<section>

...

<footer class="footer">
  <div class="site">
    
    <div>
      <p class="name">BB Assistant Company</p>
    </div>
  </div>
</footer>

</body>
```

index.html



```
...
@media (min-width: 900px) {
  .form .btn {
    width: 272px;
  }
}

/* Footer */
.footer {
  background-color: #2E5180;
}

.footer .site .name {
  color: #ffffff;
}
```

style.css

Set the footer's company name to white.

Set the footer's background color to blue.

Next, adjust the logo image's width and right margin and the company name's font size.

Like other parts, put all footer content in `<div class="container">`, making it the same width as the header content. Insert the top and bottom spaces with the padding of `<footer class="footer">`.

```
<footer class="footer">
  <div class="container">
    <div class="site">
      
      <div>
        <p class="name">BB Assistant Company</p>
      </div>
    </div>
  </div>
</footer>
```

index.html

```
/* Header's Site Name */
```

```
...
.site img {
  width: 42px;
  margin-right: 10px;
}
```

```
.site .name {
  color: #333333;
  font-size: 14px;
  font-weight: 900;
  line-height: 1.2;
}
```

```
...
```

```
/* Footer */
```

```
.footer {
  padding: 38px 0 56px;
  background-color: #2E5180;
}
```

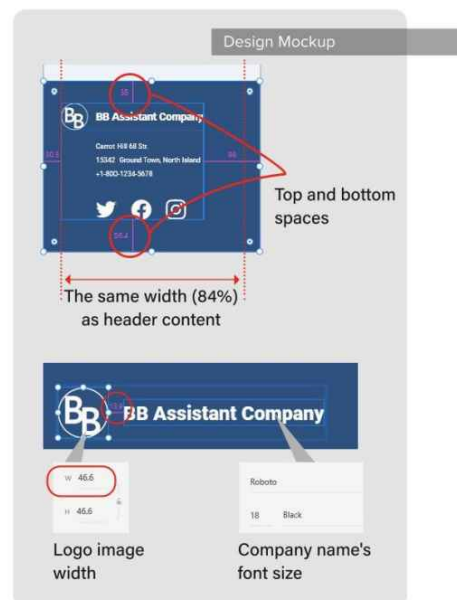
```
.footer .site img {
  width: 47px;
  margin-right: 14px;
}
```

```
.footer .site .name {
  color: #ffffff;
  font-size: 18px;
}
```

style.css



The logo image and company name get larger.



These settings for the header's logo image and company name also affect the footer's ones.

Insert the top and bottom spaces to the footer.

Set the footer's logo image width and right margin.

Set the footer's company name font size.

8-3

Adding the address

We add the address with `<p class="add">` and adjust the color, font size, line height, margin. Especially the left margin is for aligning the left side of the company name and address.

The `<div class="info">` makes them all a group as the site information. This grouping makes it easier to adjust the desktop layout.

```
<footer class="footer">
  <div class="container">
    <div class="info">
      <div class="site">
        ...
      </div>

      <p class="add">
        Carrot Hill 68 Str.<br>
        15342 Ground Town, North Island<br>
        +1-800-1234-5678
      </p>
    </div>
  </div>
</footer>
```

index.html

```
...
.footer .site .name {
  color: #ffffff;
  font-size: 18px;
}

.footer .add {
  margin: 20px 0 36px 61px;
  color: #ffffff;
  font-size: 12px;
  line-height: 2;
}
```

style.css

Set the address's margin, color, font size, line height.

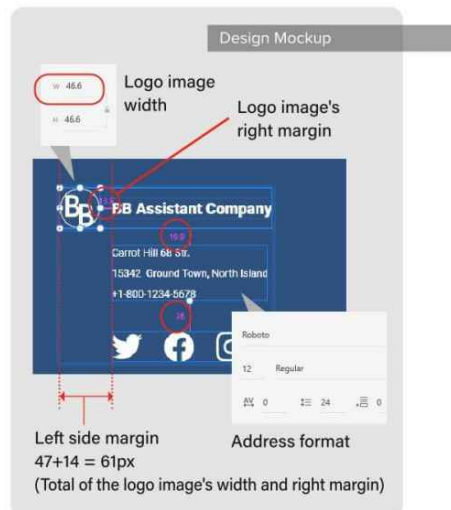


The address is displayed.



Align the left side of the company name and address.

Margin around the address.

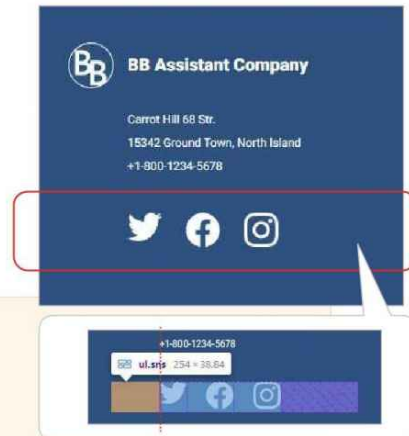


8-4

Adding the SNS menu

The SNS menu is a list of logo image links, so we create it with `<ul>`, `<li>`, `<a>`, and `<img>`.

The Flexbox of `<ul>` aligns images horizontally, and the left margin aligns the left side of the menu to the company name and address.



```
<footer class="footer">
  <div class="container">
    <div class="info">
      ...
    </div>

    <ul class="sns">
      <li><a href="#"></a></li>
      <li><a href="#"></a></li>
      <li><a href="#"></a></li>
    </ul>
  </div>
</footer>
```

index.html

```
...
.footer .add {
  margin: 20px 0 36px 61px;
  color: #ffffff;
  font-size: 12px;
  line-height: 2;
}

.footer .sns {
  display: flex;
  margin-left: 61px;
  list-style: none;
}

.footer .sns li:not(:first-child) {
  margin-left: 25px;
}

.footer .sns img {
  width: 35px;
}
```

style.css

Apply the Flexbox setting to `<ul class="sns">` and set the left margin to 61px. the `list-style` removes the default list mark.

Set the image spacing.

Set the image width.

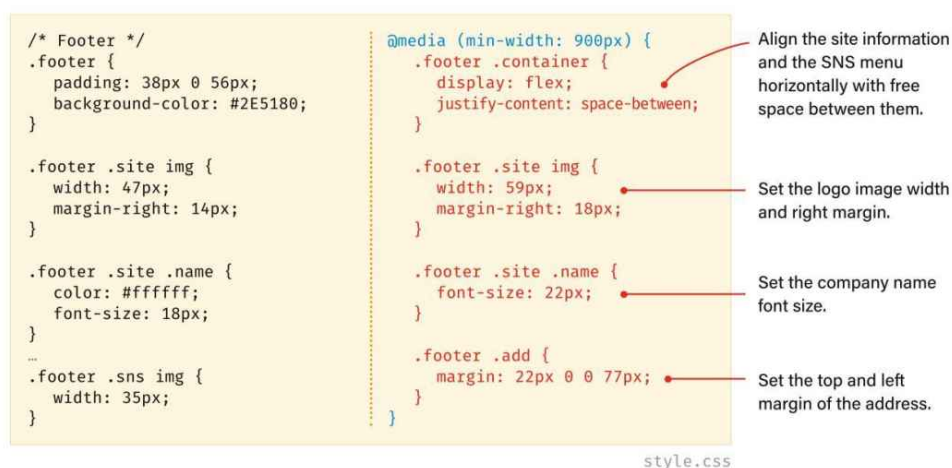
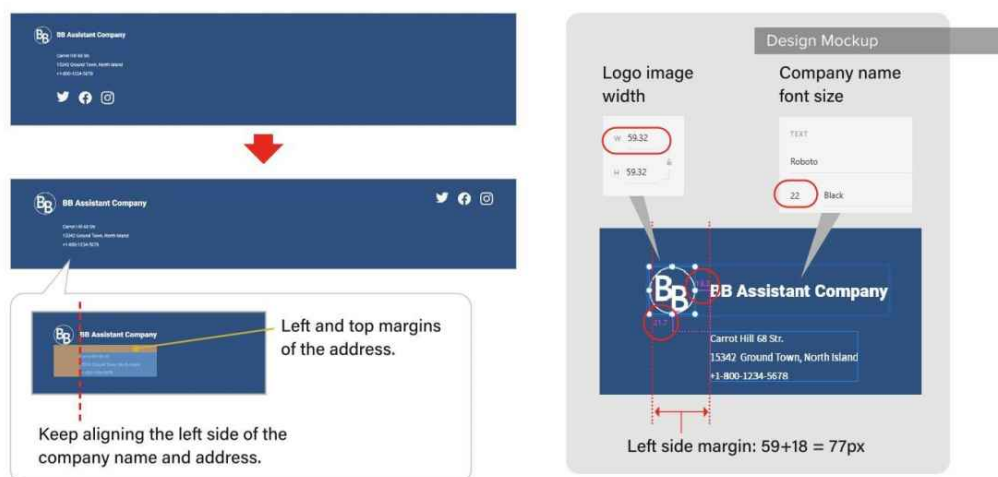


The left side margin is the same size (61px) as the address's one.

8-5

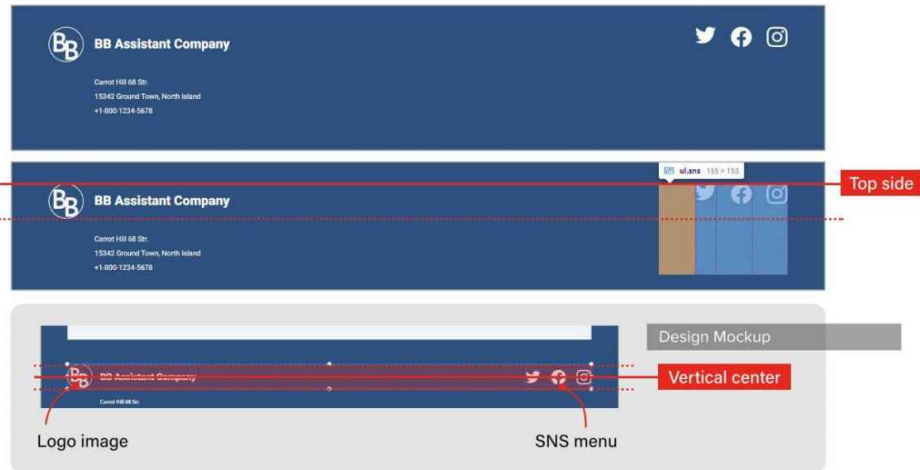
Adjusting the layout for desktop

For the desktop layout, we use Flexbox to put the site information (logo, company name, address) on the left side and the SNS menu on the right side. Make the logo and company name larger, but keep aligning the left side of the company name and address by adjusting the margin.

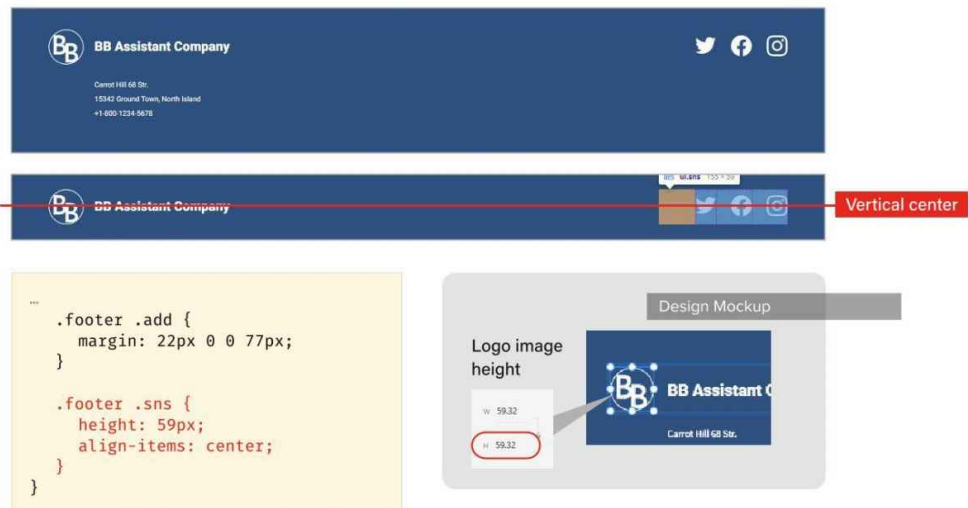


Centering vertically

Now the top side of the logo image and the SNS menu is aligned. But in the mockup, the vertical center of them is aligned.

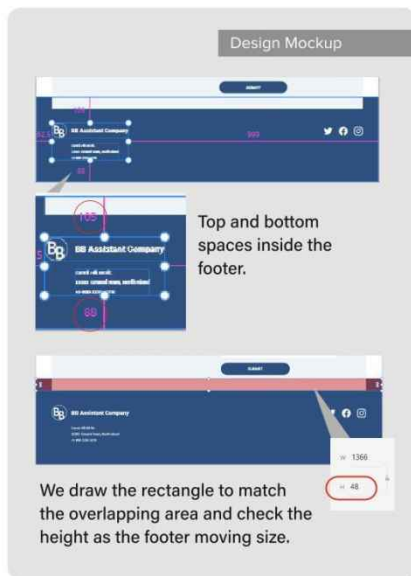


To align the vertical center of them, make the SNS menu `<ul class="sns">` the same height (59px) as the logo image and apply **align-items: center**.



Overlapping the parts

We overlap the footer with the Contact Us part. To do so, move the footer 48px upwards by the minus margin. The top and bottom padding adjust the footer height and content position. Now the footer part is complete.



Without specifying the stacking order, the footer overlaps on top. So put the Contact Us on top with **position** and **z-index** of <section class="contact">.

```
@media (min-width: 900px) {
  .contact {
    position: relative;
    z-index: 2;
  }

  .footer {
    margin-top: -48px;
    padding: 105px 0 88px;
  }

  .footer .container {
    display: flex;
  }
}
```

style.css

Specify the top margin as -48px to move the footer 48px upwards.

Set the top and bottom padding inside the footer.

Applying the **z-index:-1** to <footer class="footer"> could make the same appearance as above, but it makes links inside the footer unclickable.



STEP 9

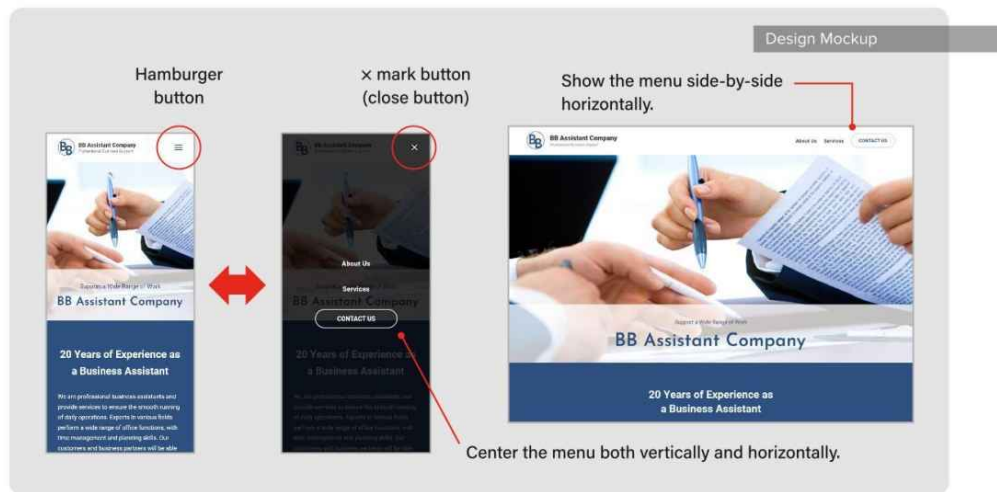
Navigation Menu

- 9-1 Creating the navigation menu
- 9-2 Adding the navigation menu as a full-screen overlay
- 9-3 Toggling the overlay with button
- 9-4 Adjusting the navigation menu for desktop

9-1

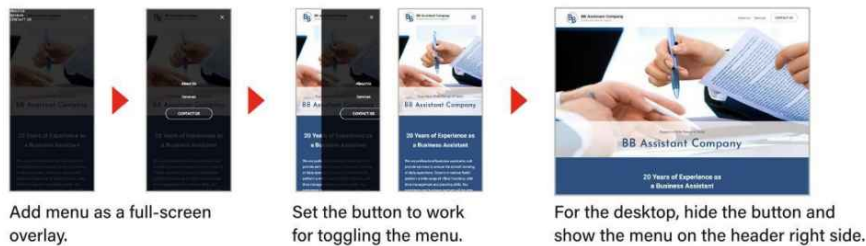
Creating the navigation menu

We create the header navigation menu. On the mobile, set the menu to show as a full-screen overlay by clicking the button. On the desktop, hide the button and place the menu side-by-side horizontally to the right side of the header.



Creation Step

The navigation button is ready as a hamburger button in STEP 3-3. So we set it to work as follows.



9-2

Adding the navigation menu as a full-screen overlay

First, add the navigation menu with `<nav class="nav">` in the header and adjust the appearance in the open state.

To style it as an overlay, fit the width and height of `<nav>` to the screen (browser window size) and set the background to transparent black color with `rgba()`.

```
<header class="header">
  <div class="container">
    <div class="site">
      ...
    </div>

    <nav class="nav">
      <ul>
        <li><a href="#">About Us</a></li>
        <li><a href="#">Services</a></li>
        <li><a href="#">CONTACT US</a></li>
      </ul>
    </nav>

    <button type="button" class="nav-button">
      <span class="sr-only">MENU</span>
    </button>
  </div>
</header>
```

index.html



The overlay is displayed.



```
/* Navigation Button */
.nav-button {
  ...
  .nav-button::before {
    transform: translateY(-5px);
    box-shadow: 0 6px currentColor;
  }
}

/* Navigation Menu on Mobile */
html, body {
  height: 100%;
  overflow: hidden;
}
```

Add this to prevent unwanted scrolling on mobile devices such as iOS and Android that is caused by setting the overlay height to 100vh.

```
.header {
  position: relative;
}

.nav {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  background: rgba(0,0,0,0.8);
  color: #ffffff;
}
```

Set the overlay background color and the menu color.

Add additional settings to prevent unwanted scrolling.

Set the overlay's position to the top left of the page.

Set the overlay's width to 100% for fitting the screen.

Set the overlay's height to 100vh for fitting the screen.

Note: If you set it to 100%, not 100vh, it will be fit the header's height.

Adjusting the menu

In the overlay, center the menu vertically and horizontally with Flexbox.

To make the third link <a> as a button, add the 'btn' class created in STEP 6-5. Set the button's border and text color to 'inherit' to inherit the white color of the parent <nav class="nav">.

```
...
<nav class="nav">
  <ul>
    <li><a href="#">About Us</a></li>
    <li><a href="#">Services</a></li>
    <li><a href="#" class="btn">
      CONTACT US</a></li>
  </ul>
</nav>
</div>
</header>
```

index.html

```
...
.nav {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  background: rgba(0,0,0,0.8);
  color: #ffffff;
  display: flex;
  justify-content: center;
  align-items: center;
}

.nav ul {
  list-style: none;
  font-weight: bold;
  text-align: center;
}

.nav li:not(:last-child) {
  margin-bottom: 40px;
}

.nav .btn {
  color: inherit;
}

/* Hero */
```

style.css

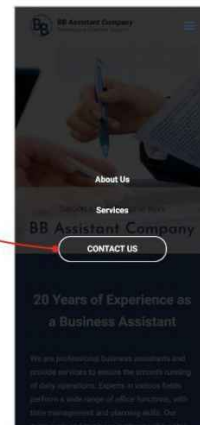
Set the menu position to the center vertically and horizontally.

Delete the menu's default list mark, set the font-weight to bold, and align the text center.

Set the space between menu items.

Set the button's border and text color.

Button style.



Adjustment result.

Design Mockup

Font weight
16 Roboto Bold

Space between menu items

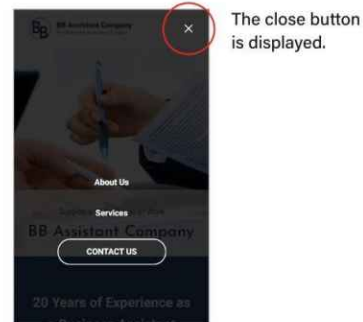


Button border

Displaying the close button

On the overlay, turn the hamburger button to the close button. We make the × mark of the close button by rotating the two hamburger lines created with the pseudo-elements of `<button class="nav-button">`.

The button won't work yet, but we set the **transition** to animate `≡` and `×` by clicking it.



```
/* Navigation Button */
.nav-button {
  box-sizing: content-box;
  padding: 0;
  outline: none;
  border: none;
  background: none;
  width: 20px;
  height: 14px;
  cursor: pointer;
  color: #2E5180;
}

.nav-button::before,
.nav-button::after {
  content: '';
  display: block;
  height: 2px;
  background-color: currentColor;
  transform: translateY(5px);
  transition: 0.3s ease-in-out;
}

.nav-button::before {
  transform: translateY(-5px);
  box-shadow: 0 6px currentColor;
}

/* Navigation Close Button */
.nav-button {
  z-index: 1000;
  color: #ffffff;
}

.nav-button::before {
  transform: translateY(1px) rotate(45deg);
  box-shadow: none;
}

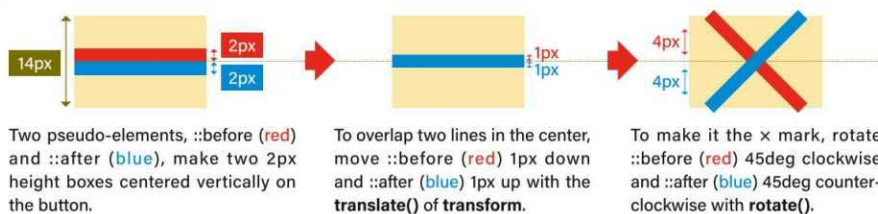
.nav-button::after {
  transform: translateY(-1px) rotate(-45deg);
}

/* Navigation Menu on Mobile */
...
```

Put the close button over the overlay by z-index, and set the line color to white.

Add the Animation setting.

Make the × mark by rotating two lines, and delete the third line created with box-shadow.

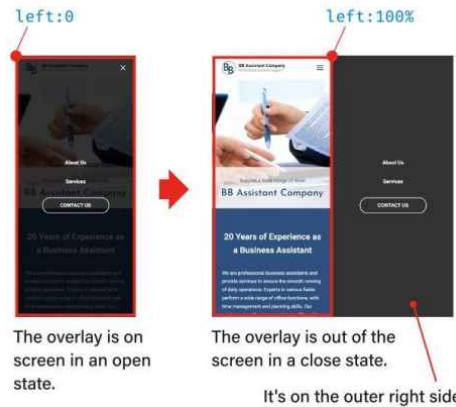


9-3

Toggling the overlay with button

We'll make the overlay open/close toggling with the button clicking. First, set the overlay's position to **left: 0** in an open state and **left: 100%** in a close state. It will slide in/out from the right side by changing the **left** value smoothly with animation.

We'll add the **'open'** class to **<html>** only when opened, so add the **.open** selector to the settings that would like to apply in an open state.



```

/* Navigation Close Button */
.open .nav-button {
  z-index: 1000;
  color: #ffffff;
}

.open .nav-button::before {
  transform: translateY(1px) rotate(45deg);
  box-shadow: none;
}

.open .nav-button::after {
  transform: translateY(-1px) rotate(-45deg);
}

/* Navigation Menu on Mobile */
html.open, .open body {
  height: 100%;
  overflow: hidden;
}

.open .form {
  display: none;
}

.open nav {
  left: 0;
}

.header {
  position: relative;
}

.nav {
  position: absolute;
  top: 0;
  left: 100%;
  width: 100%;
  height: 100vh;
  background: rgba(0,0,0,0.8);
  color: #ffffff;
  display: flex;
  justify-content: center;
  align-items: center;
  transition: left 0.5s;
}

html, body {
  overflow-x: hidden;
}

.nav ul {
  list-style: none;
  font-weight: bold;
  text-align: center;
}

.nav li:not(:last-child) {
  margin-bottom: 40px;
}

.nav .btn {
  color: inherit;
}

```

Make the close button settings apply only in an open state.

Set the overlay's position in a close state.

Set the left value to change smoothly, over a 0.5s duration.

Hide the overlay on the out of the screen.

Make the preventing scroll settings apply only in an open state.

Make the preventing scroll settings work on the iOS 12.x.

Set the overlay's position in an open state.

Adding the JavaScript

We use JavaScript to toggle between adding and removing the 'open' class from <html> with the button clicking.

Now the button can toggle the open/close overlay.

```
<!DOCTYPE html>
<html lang="en">
...

<header class="header">
  <div class="container">
    <div class="site">
      ...
    </div>

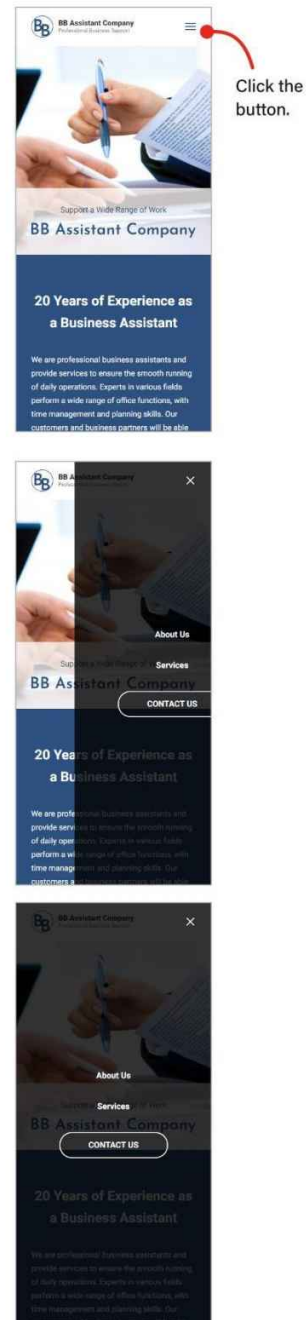
    <button type="button" class="nav-button"
      onClick="navFunc()">
      <span class="sr-only">MENU</span>
    </button>

    <nav class="nav">
      <ul>
        <li><a href="#">About Us</a></li>
        <li><a href="#">Services</a></li>
        <li><a href="#" class="btn">CONTACT US</a></li>
      </ul>
    </nav>
  </div>
</header>

...
</footer>

<script>
function navFunc() {
  document.querySelector('html').classList.toggle('open');
}
</script>
</body>
</html>
```

index.html



The overlay is opened and shows the menu. It'll be closed with the x button clicking.

9-4

Adjusting the navigation menu for desktop

For the desktop layout, hide the button and overlay, then show the menu side-by-side horizontally instead. So we apply the mobile settings only if the screen is 899px wide or less. If it's 900px wide or more, apply the desktop settings to display the menu.

Now the navigation menu is complete.



The menu is displayed side-by-side.



```
/* Navigation Menu on Mobile */
@media (max-width: 899px) {
  html.open, .open body {
    height: 100%;
    overflow: hidden;
  }
  ...
  .nav .btn {
    color: inherit;
  }
}
```

Apply the mobile settings if the screen is 899px wide or less.

```
/* Navigation Menu on Desktop */
@media (min-width: 900px) {
  .nav-button {
    display: none;
  }
}
```

Hide the button.

```
.nav ul {
  display: flex;
  align-items: center;
  list-style: none;
  color: #333333;
}
```

Align the menu items side-by-side horizontally and the vertical center of them.

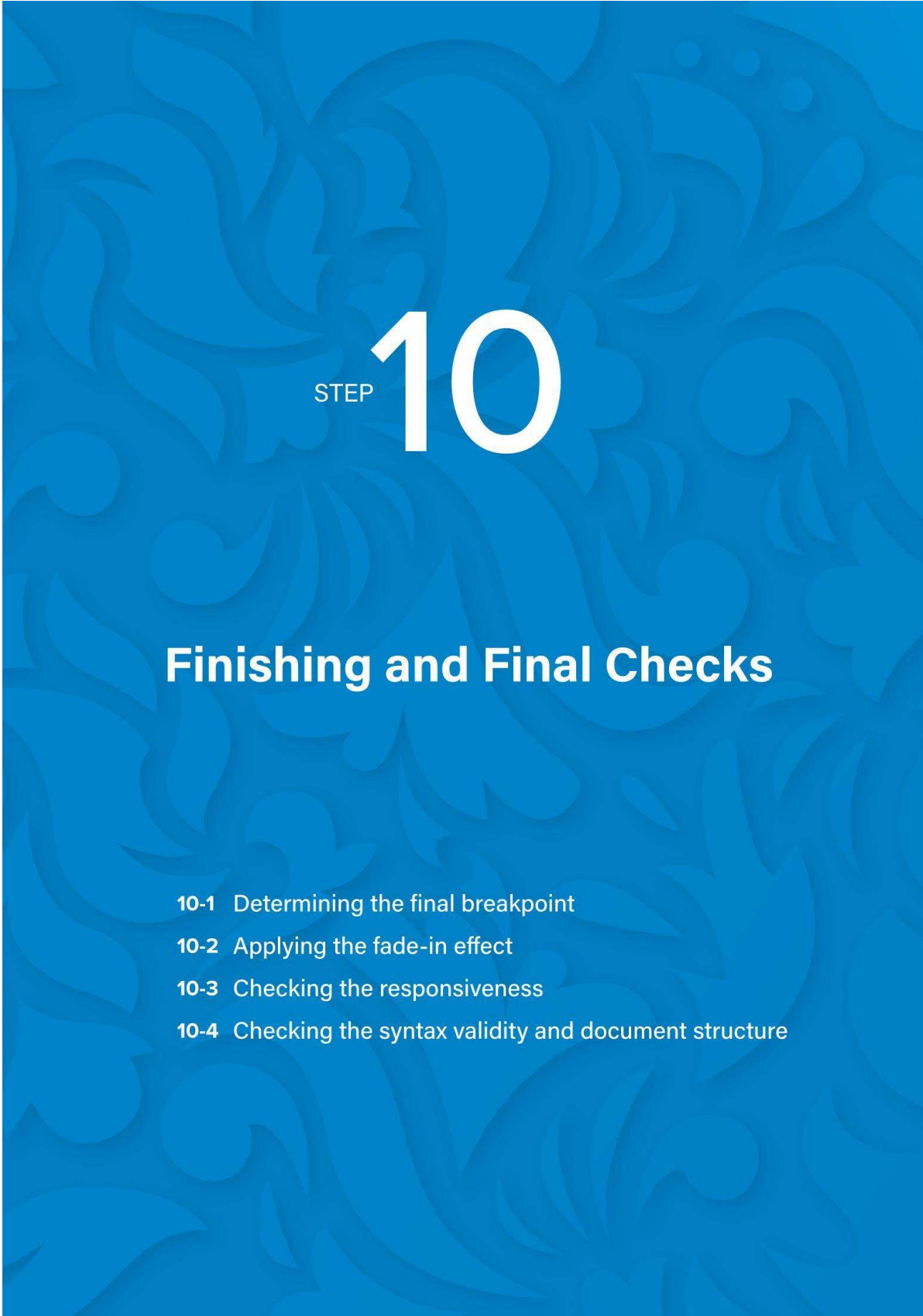
```
.nav li:not(:last-child) {
  margin-right: 23px;
}
```

Set the space between the menu items.

```
.nav .btn {
  width: 148px;
  border-color: #CDD6DD;
}
```

Set the button width and border color.

style.css



STEP 10

Finishing and Final Checks

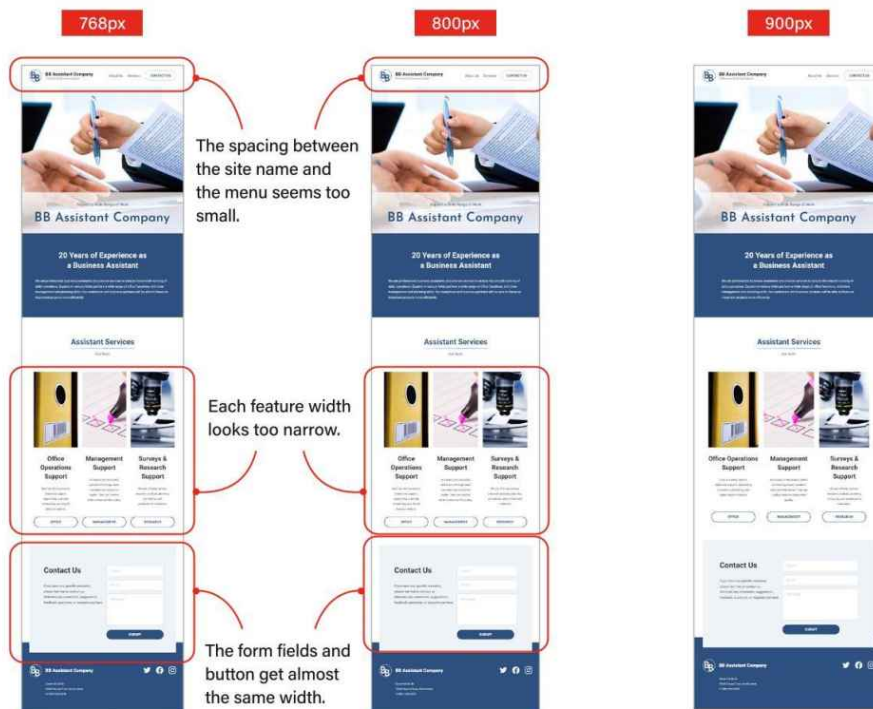
- 10-1 Determining the final breakpoint
- 10-2 Applying the fade-in effect
- 10-3 Checking the responsiveness
- 10-4 Checking the syntax validity and document structure

10-1

Determining the final breakpoint

We set the temporary breakpoint to 900px width at STEP 3-5, so determine the final width.

If we change the temporary breakpoints to 768px, 800px, 900px width, results are as follows. At the 768px or 800px, some part's space and width seems too narrow. Thus, we decide **900px** width is the final breakpoint and use current settings without changing.



There's also a way to use different breakpoints for each part, but we use a single breakpoint in this project.



10-2

Applying the fade-in effect

To apply a fade-in effect to the page on load, we use CSS animation to change the opacity value gradually.



The page is faded-in on loading.

```

...
.footer .sns {
  height: 59px;
  align-items: center;
}
}

/* Fade-in Effect */
body {
  animation: fade 5s;
}

@keyframes fade {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

```

style.css

Apply the 'fade' keyframe to <body> and set it to run over 5 seconds.

Create the 'fade' keyframe.

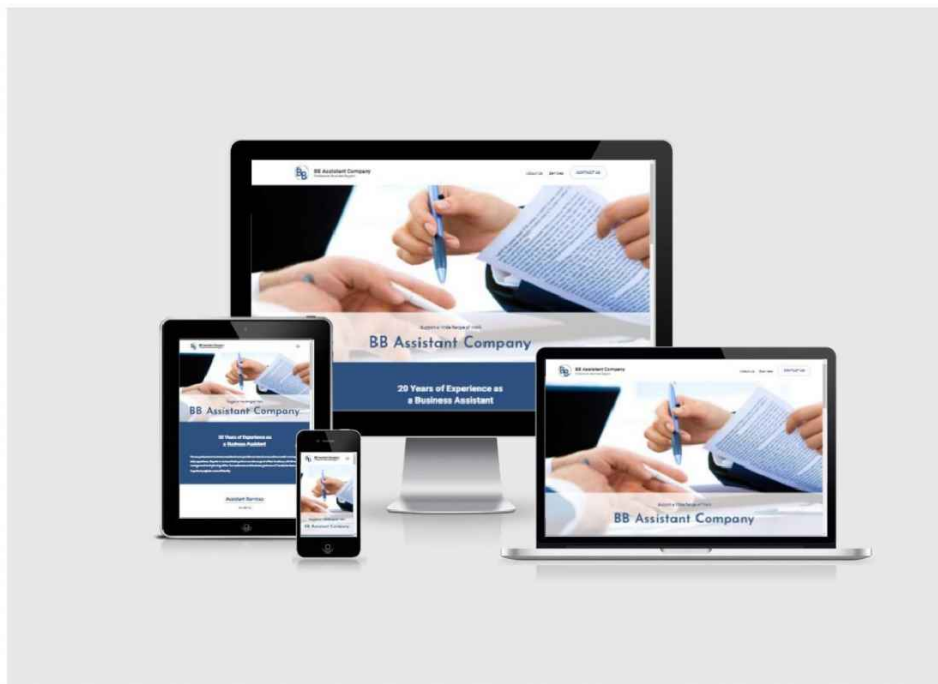
Set the opacity of the starting point (0%) to '0' — fully transparent.

Set the opacity of the ending point (100%) to '1' — fully opaque.

10-3

Checking the responsiveness

Once the page is complete, check the responsive design is working and adapting to various screen sizes.



'Am I Responsive?' shows the responsive design as above.

Am I Responsive?
<http://ami.responsivedesign.is/>

'Am I Responsive?' is an easy-to-use online tool that shows us how responsive a page is. Getting an instant visual screenshot with some device mockups helps to expect how it will look on those screen sizes.





320px



iPhone 5/SE

375px



iPhone X/XS/13 mini

414px



iPhone XR/XS Max
/11Pro Max

600px



Amazon Fire





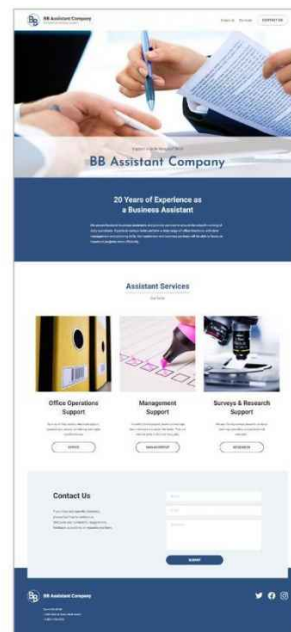
iPad (portrait)



Desktop / iPad (landscape)



Desktop / iPad Pro 12.9 (landscape)





1440px

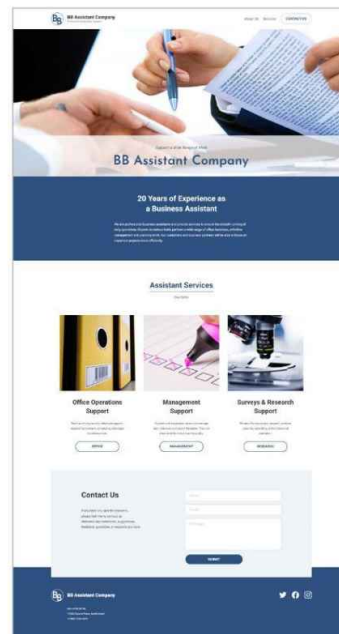
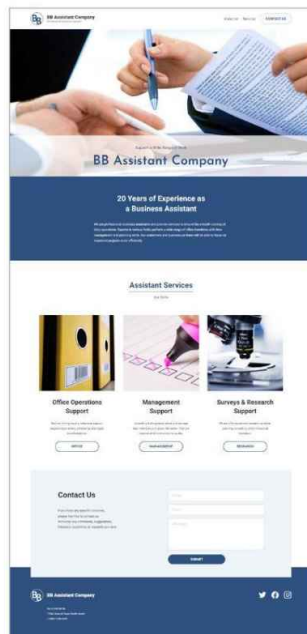


Desktop (WXGA+)

1600px



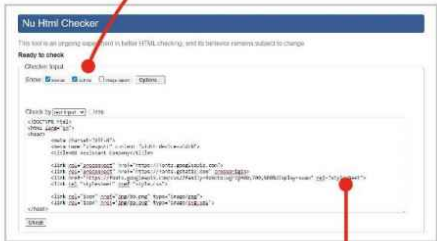
Desktop (UXGA)



10-4

Checking the syntax validity and document structure

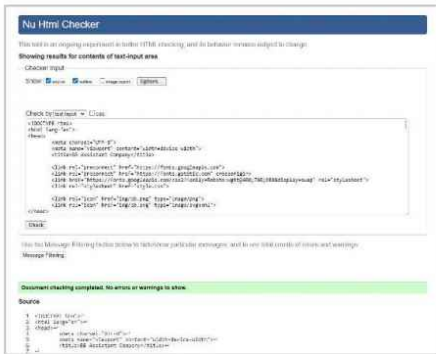
Lastly, check the HTML syntax validity and document structure of the page. We use the online validator 'Nu Html Checker,' which tracks the WHATWG HTML Standard. A green colored bar with 'No errors or warnings to show' confirms no syntax problems.



Select 'outline'.

Nu Html Checker
<https://validator.w3.org/nu/>

Select 'text input', paste HTML code, and click 'Check' button.

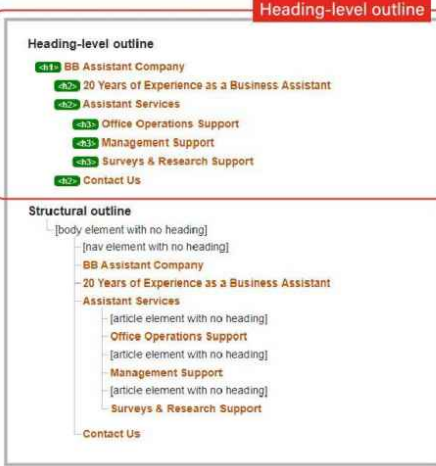


Checking result.

The outline on the bottom of the result shows the document structure.

The heading-level outline is the same as expected at STEP 1-5 (p.24). You can see the heading <h1>-<h6> makes this outline and any <div> doesn't affect it.

Now the final check is complete.



Heading-level outline

- BB Assistant Company
- 20 Years of Experience as a Business Assistant
- Assistant Services
- Office Operations Support
- Management Support
- Surveys & Research Support
- Contact Us

Structural outline

- [body element with no heading]
- [nav element with no heading]
- BB Assistant Company
- 20 Years of Experience as a Business Assistant
- Assistant Services
 - [article element with no heading]
 - Office Operations Support
 - [article element with no heading]
 - Management Support
 - [article element with no heading]
 - Surveys & Research Support
- Contact Us

Document structure (outline).